School of Computer Science and Engineering

B.Tech. First Year | Programming for Problem Solving

C Programming

Unit 1: Computer Fundamentals

Lecture Notes

Topics covered

Introduction of Computers • Classification of Computers • Anatomy of a Computer • Memory Hierarchy • Introduction to OS • Operational Overview of a CPU

Program Fundamentals: Generation and Classification of Programming Languages • Compiling, Interpreting, Loading, Linking • Developing a Program • Software Development

Algorithms: Definitions • Step-form, Pseudo-code, Flowchart • Strategy for Designing Algorithms • Structured Programming Concept

Basics of C: Overview • Developing Programs in C • Parts and Structure of a C Program • Comments • Program Statements • C Tokens • Keywords • Identifiers • Data Types • Variables • Constants • Operators and Expressions • Expression Evaluation, Precedence and Associativity • Type Conversions

Prepared as classroom lecture notes for theory preparation, programming practice and quick revision.

Contents

1. Introduction to Computers 1
- 1.1 What is a Computer? 1
- 1.2 Characteristics of a Computer 1
- 1.3 Applications of Computers 2

2. Classification of Computers 2
- 2.1 Classification by Purpose 2
- 2.2 Classification by the Type of Data Handled 2
- 2.3 Classification by Size and Capacity 3
- 2.4 Generations of Computers 3

3. Anatomy of a Computer 4
- 3.1 The Functional Units 4
- 3.2 Hardware and Software 5

4. Memory Hierarchy 5
- 4.1 Why a Hierarchy? 5
- 4.2 Levels of the Hierarchy 6
- 4.3 RAM versus ROM 6
- 4.4 Primary versus Secondary Memory 6
- 4.5 Units of Memory 7

5. Introduction to Operating Systems 7
- 5.1 What is an Operating System? 7
- 5.2 Functions of an Operating System 8
- 5.3 Types of Operating Systems 8
- 5.4 Some Important OS Terms 9

6. Operational Overview of a CPU 9
- 6.1 What the CPU Does 9
- 6.2 Important Registers 9
- 6.3 The Fetch – Decode – Execute Cycle 10
- 6.4 Clock Speed, Cores and Buses 10

7. Generation and Classification of Programming Languages 11
- 7.1 What is a Programming Language? 11
- 7.2 Generations of Programming Languages 11
- 7.3 Low-level versus High-level Languages 11
- 7.4 Classification by Programming Style (Paradigm) 12

8. Language Translators: Compiling and Interpreting 12
- 8.1 Why Translation is Needed 12
- 8.2 Assembler 12
- 8.3 Compiler 13
- 8.4 Interpreter 13
- 8.5 Compiler versus Interpreter 13

9. Loading and Linking of a Program 14
- 9.1 The Complete Translation Chain 14
- 9.2 Linking 14
- 9.3 Loading 15
- 9.4 Compiling a C Program with GCC 15

10. Developing a Program 15
- 10.1 Steps in Program Development 15
- 10.2 Types of Errors in a Program 16
- 10.3 Qualities of a Good Program 16

11. Software Development 17

11.1 What is Software Development? 17
11.2 The Software Development Life Cycle (SDLC) 17
11.3 Software Development Models 17
11.4 Characteristics of Good Software 18
12. Algorithms 18
12.1 Definition 18
12.2 Characteristics of a Good Algorithm 18
12.3 Why Write an Algorithm before Coding? 19
12.4 Examples of Simple Algorithms 19
13. Different Ways of Stating Algorithms 20
13.1 Step-form (Natural-language steps) 20
13.2 Pseudo-code 20
13.3 Flowchart 21
13.4 Comparison of the Three Forms 24
14. Strategy for Designing Algorithms 24
14.1 The Problem-Solving Approach 24
14.2 Top-down Design (Stepwise Refinement) 24
14.3 Bottom-up Design 25
14.4 Common Algorithm-design Techniques 25
14.5 Efficiency of an Algorithm 25
15. Structured Programming Concept 26
15.1 What is Structured Programming? 26
15.2 The Three Control Structures 26
15.3 Modular Programming 26
15.4 Advantages of Structured Programming 26
15.5 Structured versus Unstructured Programming 27
16. Overview of C 27
16.1 History of C 27
16.2 Features of C 28
16.3 Uses of C 28
16.4 Advantages and Disadvantages 28
17. Developing Programs in C 28
17.1 What You Need 28
17.2 Steps to Write, Compile and Run a C Program 29
17.3 Files Created during Compilation 29
17.4 Your First C Program 29
18. Parts and Structure of a C Program 30
18.1 Parts of a Simple C Program 30
18.2 General Structure of a C Program 31
18.3 A Program Showing All the Sections 32
18.4 The main() Function 32
18.5 Header Files 33
19. Comments and Program Statements 33
19.1 Comments 33
19.2 Program Statements 34
20. C Tokens, Keywords and Identifiers 35
20.1 C Tokens 35
20.2 Keywords 35
20.3 Identifiers 36
21. Data Types 37
21.1 What is a Data Type? 37
21.2 Primary (Basic / Fundamental) Data Types 37

21.3 Type Modifiers 38
21.4 Derived and User-defined Data Types 38
21.5 The void Type 38
21.6 Program: Size of Data Types 39
21.7 Program: Using Different Data Types 40
21.8 Common Mistakes with Data Types 40
22. Variables and Constants 41
22.1 Variables 41
22.2 Constants 42
22.3 Symbolic Constants: #define and const 42
22.4 Program: Variables and Constants 43
23. Operators 44
23.1 Arithmetic Operators 44
23.2 Relational Operators 45
23.3 Logical Operators 45
23.4 Assignment Operators 45
23.5 Increment and Decrement Operators 46
23.6 Conditional (Ternary) Operator 46
23.7 Bitwise Operators 46
23.8 Special Operators 47
23.9 Program: Arithmetic, Relational and Logical Operators 47
23.10 Program: Increment, Decrement, Assignment and Conditional Operators 48
24. Expressions and Expression Evaluation 49
24.1 What is an Expression? 49
24.2 How an Expression is Evaluated 50
24.3 Integer Expressions and Integer Division 50
24.4 Real (Floating-point) and Mixed-mode Expressions 51
24.5 Program: Expression Evaluation 51
24.6 Order of Evaluation of Operands (a caution) 52
25. Operator Precedence and Associativity 52
25.1 Precedence 52
25.2 Associativity 52
25.3 Precedence and Associativity Table 52
25.4 Worked Examples 53
25.5 Program: Precedence and Associativity 54
26. Type Conversions 55
26.1 Why Conversion is Needed 55
26.2 Implicit Type Conversion (Automatic / Coercion) 55
26.3 Explicit Type Conversion (Type Casting) 56
26.4 Program: Implicit and Explicit Conversion 57
26.5 Implicit versus Explicit Conversion 57
26.6 Common Mistakes 58
27. Common Mistakes and Exam-oriented Questions 58
27.1 Ten Common Beginner Mistakes in C 58
27.2 Short-answer Questions (2 marks) 59
27.3 Long-answer Questions (5 to 10 marks) 59
27.4 Predict-the-output Practice 60
28. Quick Revision Sheet 60
28.1 Computer Fundamentals 60
28.2 Program Fundamentals ..... **Error! Bookmark not defined.**
28.3 Algorithms and Structured Programming ..... **Error! Bookmark not defined.**
28.4 Basics of C ..... **Error! Bookmark not defined.**
28.5 Formula and Fact Card 61

PART A | COMPUTER FUNDAMENTALS

1. Introduction to Computers

1.1 What is a Computer?

A **computer** is an electronic device that accepts data from the user (input), processes it according to a set of instructions (a program), stores the data and the results, and produces useful information (output). The word comes from the Latin word *computare*, meaning "to calculate", but a modern computer does far more than arithmetic: it edits documents, plays videos, connects to the internet and controls machines.

The most important idea for a programmer is that a computer does **exactly** what it is told, in the order it is told, and nothing more. It has no common sense. Every task, however small, must be broken into precise steps. Writing those steps in a language the computer understands is called **programming**.

Data versus Information

Data	Information
Raw facts and figures that have not been processed (e.g. 78, 65, 90)	Processed data that has meaning and is useful for decisions (e.g. "average marks = 77.6")
Input to the computer	Output from the computer
Has no meaning on its own	Has meaning and context

The Input – Process – Output (IPO) cycle

Every computing task follows the same pattern: **Input** (data goes in through keyboard, mouse, scanner, sensor, file), **Processing** (the CPU carries out instructions on the data), **Storage** (data and results are kept in memory) and **Output** (results appear on the screen, printer or speaker). Keep this cycle in mind; every C program you write will read input, process it and print output.

1.2 Characteristics of a Computer

- **Speed** – a modern processor executes billions of simple instructions per second. Work that takes a human hours is finished in a fraction of a second.
- **Accuracy** – a computer does not make calculation mistakes. Errors that do occur are caused by wrong data or wrong programs (**GIGO: Garbage In, Garbage Out**).
- **Storage** – huge amounts of data can be stored and retrieved instantly (terabytes on a single disk).
- **Diligence** – a computer never gets tired or bored; it performs the millionth calculation as accurately as the first.
- **Versatility** – the same machine can do accounting, play music, run a game and control a robot, simply by changing the program.
- **Automation** – once a program is started, the computer carries out the whole job without human intervention.
- **Reliability** – results are consistent; the same input with the same program always gives the same output.

Limitations of a computer

- **No intelligence of its own** – it cannot decide what to do unless a program tells it.
- **No feelings or common sense** – it cannot judge whether an instruction is sensible.
- **Depends on the user** – wrong input or a wrong program produces wrong output.

- **Needs electricity** and a suitable environment (heat and dust can damage it).

1.3 Applications of Computers

Computers are used in **education** (online classes, digital libraries), **banking** (ATMs, net banking, UPI), **healthcare** (patient records, scanning machines), **business** (accounting, inventory, e-commerce), **science and engineering** (simulations, CAD design, weather forecasting), **communication** (e-mail, video calls, social media), **entertainment** (games, films, streaming), **government** (Aadhaar, tax filing, railway reservation) and **defence and space research** (ISRO mission control).

Key points for the exam

- Computer = electronic device that performs Input, Processing, Storage and Output under program control.
- Data is raw; information is processed data with meaning.
- Main characteristics: speed, accuracy, storage, diligence, versatility, automation.
- GIGO – the computer is only as good as the data and program given to it.

2. Classification of Computers

Computers can be classified in three common ways: by the **purpose** they serve, by the **type of data** they handle, and by their **size and processing capacity**. A fourth way, by **generation**, describes how computer technology has evolved over time.

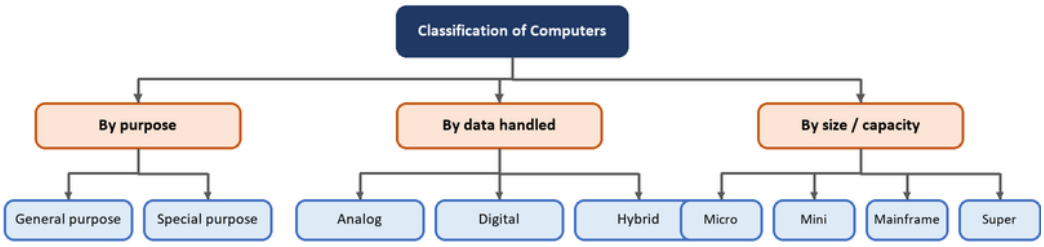


Figure 2.1 – Three ways of classifying computers

2.1 Classification by Purpose

- **General-purpose computers** are designed to do many different kinds of jobs. Desktops, laptops and smartphones are general purpose; the same machine can run a browser, a spreadsheet or a game.
- **Special-purpose computers** are built for one specific task and cannot easily be used for anything else. Examples: the controller inside a washing machine, a traffic-signal controller, an aircraft autopilot, an ATM.

2.2 Classification by the Type of Data Handled

Type	How it works	Examples
Analog computer	Works on continuously varying physical quantities such as voltage, temperature, pressure or speed. Results are approximate.	Speedometer, thermometer, old voltage meters, slide rule

Type	How it works	Examples
Digital computer	Works on discrete data represented as binary digits (0 and 1). Very accurate. This is the kind of computer we program in C.	Desktop, laptop, smartphone, calculator
Hybrid computer	Combines both: an analog part measures a physical quantity and a digital part processes it.	ICU patient monitors, petrol pump, missile guidance systems

2.3 Classification by Size and Capacity

Type	Description	Typical use / example
Microcomputer (Personal Computer)	Smallest and cheapest. Built around a single microprocessor chip. Used by one person at a time.	Desktop, laptop, tablet, smartphone, Raspberry Pi
Minicomputer (Mid-range)	More powerful than a PC; supports many users at once (multi-user). Sits between micro and mainframe.	Departmental servers in a university or a factory
Mainframe computer	Very large, very powerful; handles thousands of users and enormous data volumes with high reliability.	Banks, insurance companies, railway reservation, census processing; IBM zSeries
Supercomputer	The fastest and most expensive; thousands of processors working in parallel on one problem. Speed measured in petaflops.	Weather forecasting, nuclear research, drug design; India's PARAM series, AIRAWAT

2.4 Generations of Computers

The word *generation* describes a major change in the technology used to build computers. Each generation brought smaller size, higher speed, lower cost and less power consumption.

Generation	Period	Main technology	Features and examples
First	1940–1956	Vacuum tubes	Huge, produced great heat, unreliable, machine language only. ENIAC, UNIVAC-1
Second	1956–1963	Transistors	Smaller, faster, cheaper; assembly language and early high-level languages (FORTRAN, COBOL). IBM 1401
Third	1964–1971	Integrated Circuits (IC)	Many transistors on one chip; operating systems, keyboards and monitors appear. IBM 360
Fourth	1971–present	Microprocessor (VLSI)	Entire CPU on one chip; personal computers, GUI, networks and the internet. Intel 4004, IBM PC, today's laptops
Fifth	present and future	Artificial Intelligence, ULSI, parallel processing	Natural-language understanding, expert systems, robotics, quantum computing research

Remember

- Analog = continuous quantities; Digital = discrete 0/1 data; Hybrid = both.
- Order by size and power: Micro < Mini < Mainframe < Super.
- Generations: Vacuum tube → Transistor → IC → Microprocessor → AI.

3. Anatomy of a Computer

The *anatomy* of a computer means its internal organisation: the parts it is made of and how they work together. Every digital computer, from a smartwatch to a supercomputer, follows the same basic block diagram proposed by John von Neumann in 1945.

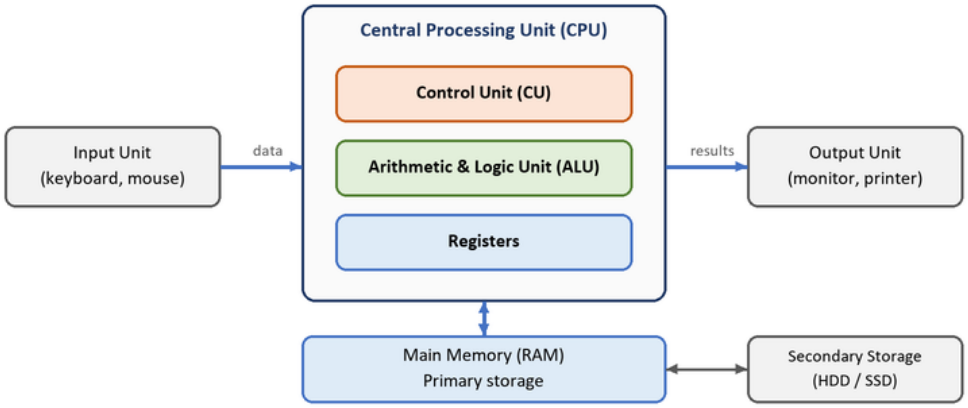


Figure 3.1 – Block diagram of a computer (von Neumann architecture)

3.1 The Functional Units

Input unit

The input unit accepts data and instructions from the outside world and converts them into a form (binary) that the computer can understand. Examples: keyboard, mouse, touch screen, scanner, microphone, camera, bar-code reader.

Central Processing Unit (CPU)

The CPU is the **brain** of the computer. It executes program instructions and controls all the other units. It has three parts:

- **Control Unit (CU)** – reads instructions from memory, decodes them and sends control signals to the other units telling them what to do and when. It does no calculation itself; it is the traffic policeman of the computer.
- **Arithmetic and Logic Unit (ALU)** – performs all arithmetic operations (+, −, ×, ÷) and logical operations (comparisons such as greater-than, equal-to, AND, OR, NOT).
- **Registers** – a small number of very fast storage locations inside the CPU that hold the data and instructions currently being worked on.

Memory unit (primary storage)

The memory unit holds the program being executed, the data it works on and the intermediate results. It is directly connected to the CPU. The main memory is **RAM** (Random Access Memory), which is fast but *volatile*: its contents vanish when the power is switched off. **ROM** (Read Only Memory) holds the permanent start-up instructions (BIOS/firmware) and is non-volatile.

Output unit

The output unit converts the processed results from binary form into a form humans can understand. Examples: monitor (soft copy), printer (hard copy), speakers, projector, plotter.

Secondary storage

Because RAM is small and volatile, programs and data are kept permanently on **secondary storage** such as a hard disk (HDD), solid-state drive (SSD), pen drive or DVD. Before a program can run, it is copied from secondary storage into RAM.

3.2 Hardware and Software

Hardware	Software
The physical, touchable parts of the computer: CPU, RAM, keyboard, monitor, hard disk.	The set of programs (instructions) that tell the hardware what to do: Windows, MS Word, a C compiler, your C program.
Manufactured in a factory; wears out.	Written by programmers; does not wear out but may become outdated.
Cannot do anything without software.	Cannot run without hardware.

Types of software

- **System software** – manages the hardware and provides a platform for other programs. Examples: operating systems (Windows, Linux), device drivers, language translators (compilers, interpreters, assemblers), utilities (antivirus, disk cleanup).
- **Application software** – helps the user perform a specific task. Examples: MS Word, Excel, Chrome, WhatsApp, Tally, a railway reservation program.
- **Firmware** – software permanently stored on a ROM chip inside a device (e.g. the BIOS of a PC, the program inside a TV remote).

Key points

- Five functional units: Input, CPU (CU + ALU + Registers), Memory, Output, Secondary storage.
- CU controls, ALU calculates, registers hold the current data.
- RAM is volatile primary memory; the hard disk / SSD is non-volatile secondary storage.
- Hardware = physical parts; Software = programs. System software runs the machine, application software serves the user.

4. Memory Hierarchy

4.1 Why a Hierarchy?

The ideal memory would be extremely fast, extremely large and extremely cheap. No single technology gives all three: the fastest memory (registers, cache) is very expensive per byte and therefore small, while the cheapest memory (hard disk, tape) is very slow. Computer designers therefore arrange **several levels** of memory, from a tiny amount of very fast memory close to the CPU down to huge amounts of slow, cheap storage far from it. This arrangement is the **memory hierarchy**.

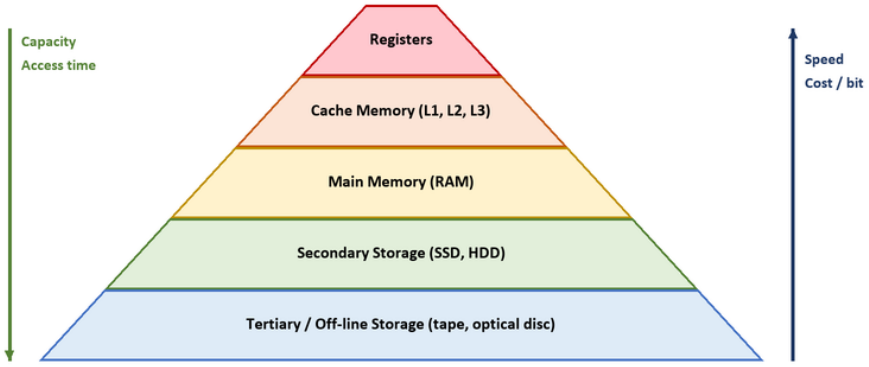


Figure 4.1 – The memory hierarchy: speed and cost increase towards the top, capacity increases towards the bottom

4.2 Levels of the Hierarchy

Level	Where	Typical size	Access time	Volatile?
Registers	Inside the CPU	A few hundred bytes	< 1 ns	Yes
Cache (L1, L2, L3)	On the CPU chip	KB to a few MB	1–10 ns	Yes
Main memory (RAM)	Motherboard	4–64 GB	50–100 ns	Yes
Secondary storage (SSD / HDD)	Inside the computer	256 GB – several TB	0.1–10 ms	No
Tertiary / off-line (tape, DVD, cloud backup)	Removable / remote	Unlimited	seconds or more	No

- **Registers** hold the operands and results of the instruction being executed right now.
- **Cache memory** is a small, very fast memory that keeps copies of the data and instructions the CPU has used recently or is likely to need next. Because programs tend to reuse the same data (the *principle of locality*), most memory requests are satisfied by the cache, and the slower RAM is accessed less often.
- **Main memory (RAM)** holds the operating system, the running programs and their data.
- **Secondary storage** keeps files, programs and the operating system permanently.
- **Tertiary / off-line storage** is used for backups and archives.

4.3 RAM versus ROM

RAM (Random Access Memory)	ROM (Read Only Memory)
Read and write memory: the CPU can both read from and write to it.	Normally read only: contents are written once by the manufacturer.
Volatile: contents are lost when power is switched off.	Non-volatile: contents remain when power is off.
Holds the programs and data currently in use.	Holds permanent start-up code (BIOS / firmware).
Larger (GBs) and faster.	Smaller (MBs).
Types: SRAM (used for cache) and DRAM (used for main memory).	Types: PROM, EPROM, EEPROM, Flash.

4.4 Primary versus Secondary Memory

Primary memory	Secondary memory

Primary memory	Secondary memory
Directly accessed by the CPU.	Not directly accessed by the CPU; data is first moved to RAM.
Fast, small, expensive per byte.	Slow, large, cheap per byte.
Mostly volatile (RAM).	Non-volatile (HDD, SSD, pen drive, DVD).
Also called main memory or internal memory.	Also called auxiliary, backing or external storage.

4.5 Units of Memory

All data inside a computer is stored as **bits** (binary digits, 0 or 1). Eight bits make one **byte**, which is enough to store one character such as 'A'. Larger units are built from bytes:

Unit	Symbol	Equal to	Approximate value
Bit	b	0 or 1	smallest unit
Nibble	–	4 bits	half a byte
Byte	B	8 bits	one character
Kilobyte	KB	1024 bytes (2^{10})	≈ one thousand bytes
Megabyte	MB	1024 KB (2^{20} bytes)	≈ one million bytes
Gigabyte	GB	1024 MB (2^{30} bytes)	≈ one billion bytes
Terabyte	TB	1024 GB (2^{40} bytes)	≈ one trillion bytes
Petabyte	PB	1024 TB (2^{50} bytes)	≈ 10^{15} bytes

Key points

- Hierarchy (fast → slow): Registers → Cache → RAM → SSD/HDD → Tape/optical.
- Going down the hierarchy: capacity and access time increase, cost per bit and speed decrease.
- Cache works because of locality of reference: programs reuse nearby data and instructions.
- 1 byte = 8 bits; 1 KB = 1024 bytes; 1 MB = 1024 KB; 1 GB = 1024 MB.

5. Introduction to Operating Systems

5.1 What is an Operating System?

An **operating system (OS)** is the system software that manages all the hardware and software resources of a computer and provides a convenient interface between the user and the hardware. It is the first program loaded when the computer starts and it remains in memory until the computer is switched off. Without an OS a computer is just a collection of electronic parts; you cannot even open a file.

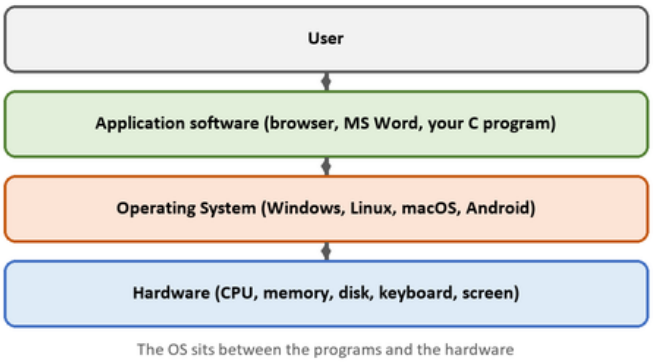


Figure 5.1 – The operating system as an interface between the user, application programs and hardware

Common operating systems: **Windows**, **Linux** (Ubuntu, Fedora), **macOS**, **Android** and **iOS** for phones, and **UNIX** on servers. The C language itself was created to write the UNIX operating system, and Linux is written almost entirely in C.

5.2 Functions of an Operating System

Function	What it means
Process management	Creates, schedules and terminates processes (running programs); decides which program gets the CPU and for how long.
Memory management	Allocates RAM to programs, keeps track of which parts are free and protects one program's memory from another.
File management	Organises data into files and folders, controls creation, reading, writing and deletion, and enforces permissions.
Device management	Controls input/output devices through device drivers; handles printing, disk access, network cards.
User interface	Provides a Command Line Interface (CLI, e.g. Linux terminal) or a Graphical User Interface (GUI, e.g. Windows desktop).
Security and protection	User accounts, passwords, access rights; prevents unauthorised access to data.
Error detection	Detects hardware and software errors and takes corrective action or reports them.

5.3 Types of Operating Systems

Type	Description	Example
Batch OS	Jobs with similar needs are grouped into batches and executed one after another without user interaction.	Early IBM mainframes, payroll processing
Multiprogramming OS	Several programs are kept in memory; when one waits for I/O the CPU switches to another, so the CPU is never idle.	Early UNIX
Time-sharing (multitasking) OS	The CPU time is divided into small slices and shared among many users or programs, so each feels it has the computer to itself.	Windows, Linux, macOS
Real-time OS (RTOS)	Must respond to inputs within a strict, guaranteed time limit. Used where a late answer is a wrong answer.	Aircraft control, pacemakers, robots, VxWorks
Distributed OS	Manages a group of separate computers connected by a network so that they appear as a single system.	Cluster and cloud systems

Type	Description	Example
Network OS	Runs on a server and lets client computers share files, printers and other resources.	Windows Server, Linux servers
Mobile OS	Designed for phones and tablets: touch interface, power saving.	Android, iOS

5.4 Some Important OS Terms

- **Kernel** – the core part of the OS that stays in memory all the time and directly controls the hardware.
- **Shell / command interpreter** – the part that accepts commands from the user (e.g. the Windows Command Prompt, the Linux bash shell).
- **Booting** – the process of starting the computer: the ROM (BIOS/UEFI) checks the hardware and loads the OS from disk into RAM.
- **Process** – a program in execution. **Multitasking** – running several processes apparently at the same time.
- **Device driver** – a small program that lets the OS talk to a particular hardware device (printer driver, graphics driver).

Key points

- OS = resource manager + user interface; it is system software.
- Main functions: process, memory, file and device management, user interface, security.
- Types: batch, multiprogramming, time-sharing, real-time, distributed, network, mobile.
- Kernel is the heart of the OS; booting loads the OS into RAM at start-up.

6. Operational Overview of a CPU

6.1 What the CPU Does

The CPU (Central Processing Unit, also called the *processor*) executes the instructions of a program one after another. A program stored in memory is just a long list of very simple instructions such as "copy this number into a register", "add these two registers" or "jump to instruction 40 if the result was zero". The CPU repeats a fixed sequence of steps for every instruction; this sequence is called the **instruction cycle** or the **fetch–decode–execute cycle**.

6.2 Important Registers

Register	Purpose
Program Counter (PC)	Holds the memory address of the **next** instruction to be fetched. It is incremented after every fetch.
Instruction Register (IR)	Holds the instruction that is currently being decoded and executed.
Memory Address Register (MAR)	Holds the address of the memory location that is to be read or written.
Memory Data Register (MDR / MBR)	Holds the data that has just been read from memory or is about to be written to memory.
Accumulator (ACC)	General-purpose register that holds the intermediate results of ALU operations.
Status / Flag register	Stores the result conditions of the last operation: zero, negative, carry, overflow. Used for decisions (if, loops).

6.3 The Fetch – Decode – Execute Cycle

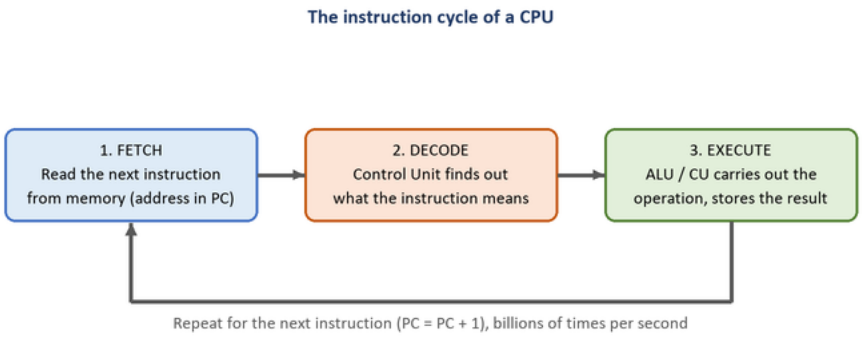


Figure 6.1 – The instruction cycle repeated by the CPU for every instruction

1. **Fetch** – The Control Unit copies the address in the PC into the MAR, reads the instruction at that address from memory into the MDR and then into the IR. The PC is incremented so that it points to the next instruction.
2. **Decode** – The Control Unit examines the bit pattern in the IR to find out what operation is required (add, compare, jump, load, store) and which registers or memory locations are involved.
3. **Execute** – The required operation is carried out: the ALU performs the calculation, or data is moved between memory and registers, or the PC is changed (for a jump). Any result is stored in a register or in memory.
4. The cycle repeats with the next instruction, billions of times every second, until the program ends.

6.4 Clock Speed, Cores and Buses

- **Clock** – a tiny crystal oscillator that produces a steady stream of pulses; every step of the CPU happens on a clock pulse. **Clock speed** is measured in hertz; a 3 GHz processor has 3 billion clock cycles per second. A higher clock speed generally means a faster CPU (for the same design).
- **Cores** – a modern processor chip contains several independent CPUs, called cores (dual-core, quad-core, octa-core). Each core can run a separate program at the same time.
- **Word size** – the number of bits the CPU handles at once: 32-bit or 64-bit processors. A 64-bit CPU can process larger numbers and address more memory.
- **Buses** – groups of parallel wires that carry information between the CPU, memory and I/O devices. The **address bus** carries the memory address, the **data bus** carries the actual data, and the **control bus** carries control signals such as read and write.
- **Instruction set** – the complete list of instructions a particular CPU can understand (e.g. Intel x86, ARM). A program compiled for one instruction set will not run on a CPU with a different one; this is why C code must be *compiled* separately for each type of machine.

Key points

- The CPU repeats Fetch → Decode → Execute for every instruction.
- PC holds the address of the next instruction; IR holds the current instruction; ACC holds results.
- Clock speed (GHz) and number of cores decide how fast a CPU is.
- Three buses: address, data and control.

PART B | PROGRAM FUNDAMENTALS

7. Generation and Classification of Programming Languages

7.1 What is a Programming Language?

A **program** is a set of instructions that tells the computer how to perform a task. A **programming language** is a formal language with its own vocabulary (keywords), grammar (syntax) and meaning (semantics) that is used to write programs. Just as English has rules of spelling and grammar, every programming language has rules that must be followed exactly, otherwise the computer cannot understand the program.

The computer's own language is **machine language**: patterns of 0s and 1s. Humans find this very hard to write, so over the years languages have been created that are closer and closer to English. These languages are grouped into **generations**.

7.2 Generations of Programming Languages

Generation	Name	Description	Examples
1GL	Machine language	Instructions written directly in binary (0s and 1s). The only language the CPU understands directly. Extremely difficult to write, read and debug; different for every type of CPU.	10110000 01100001
2GL	Assembly language	Uses short English-like mnemonics (ADD, MOV, SUB) instead of binary codes. Needs an assembler to convert it to machine code. Still machine-dependent.	MOV AL, 61h ADD AX, BX
3GL	High-level languages	English-like statements, independent of the machine. One statement does the work of many machine instructions. Needs a compiler or interpreter.	C, C++, Java, Python, FORTRAN, COBOL, BASIC
4GL	Very high-level languages	Closer to human language; the programmer says **what** is wanted, not how to do it. Mostly used for databases and reports.	SQL, MATLAB, report generators
5GL	Logic / AI languages	Based on constraints and logic; the computer works out the solution from the problem description. Used in artificial intelligence.	Prolog, LISP, Mercury

7.3 Low-level versus High-level Languages

Machine language and assembly language are called **low-level languages** because they are close to the hardware. Languages from the third generation onward are **high-level languages** because they are close to the way humans think and write.

Low-level language (machine, assembly)	High-level language (C, Java, Python)
Close to the hardware; uses binary codes or mnemonics.	Close to English; uses words like if, while, print.
Machine dependent: a program written for one CPU will not run on another.	Machine independent (portable): the same program can be compiled for any CPU.
Difficult to learn, write, read and debug.	Easy to learn, write, read and debug.
Programs are long: one line = one machine instruction.	Programs are short: one statement = many machine instructions.

Low-level language (machine, assembly)	High-level language (C, Java, Python)
Fastest execution and direct control over the hardware.	Slightly slower because a translator converts it to machine code.
Needs an assembler (for assembly language).	Needs a compiler or an interpreter.
Used for device drivers, embedded firmware, the innermost parts of an OS.	Used for almost all application and system software.

C is often called a **middle-level language** because it has the ease of a high-level language but also allows low-level operations such as direct memory access through pointers and bitwise operators. That is why operating systems, compilers and embedded software are written in C.

7.4 Classification by Programming Style (Paradigm)

Paradigm	Idea	Examples
Procedural (imperative)	The program is a sequence of instructions grouped into procedures (functions). Focus is on **how** the task is done, step by step.	C, Pascal, FORTRAN, BASIC
Object-oriented	The program is organised around **objects** that combine data and the functions that work on it (classes, inheritance, polymorphism).	C++, Java, Python, C#
Functional	Computation is treated as evaluation of mathematical functions; avoids changing state.	Haskell, LISP, Scala
Logic / declarative	The programmer states facts and rules; the system finds the answer.	Prolog, SQL
Scripting	Small programs, usually interpreted, that automate tasks or glue other programs together.	Python, JavaScript, Bash, PHP

Key points

- 1GL machine → 2GL assembly → 3GL high-level (C) → 4GL (SQL) → 5GL (Prolog).
- Low-level = machine dependent, fast, hard; High-level = portable, easy, needs a translator.
- C is a procedural, middle-level, compiled language.

8. Language Translators: Compiling and Interpreting

8.1 Why Translation is Needed

The CPU understands only machine language. A program written in C, Java or Python (the **source code**) must therefore be converted into machine code (the **object code** or **executable**) before the CPU can run it. The system software that does this conversion is called a **language translator** (or language processor). There are three kinds: the assembler, the compiler and the interpreter.

8.2 Assembler

An **assembler** translates an assembly-language program (mnemonics such as MOV, ADD) into machine code. Each assembly instruction is converted to exactly one machine instruction. Examples: MASM, NASM, TASM.

8.3 Compiler

A **compiler** reads the **entire** high-level source program, checks it for errors, and translates the whole program into machine code (an object file) in one go. The translated program is saved as an executable file (for example **hello.exe**) which can be run again and again without recompiling. If the source contains errors, the compiler lists all of them and produces no executable; the programmer must correct the errors and compile again.

Compilation happens in phases: **lexical analysis** (breaking the text into tokens), **syntax analysis** (checking grammar), **semantic analysis** (checking meaning, such as type mismatches), **code optimisation** and **code generation**. You do not need the details in Unit 1, only the idea that a compiler translates the whole program before execution. Examples of C compilers: **GCC** (GNU Compiler Collection), **Clang**, **Microsoft Visual C++**, **Turbo C**.

8.4 Interpreter

An **interpreter** translates and executes the source program **one statement at a time**. It reads a line, converts it to machine code, runs it, then moves to the next line. No separate executable file is produced; the interpreter must be present every time the program runs. If an error is found in line 20, lines 1 to 19 have already been executed and the interpreter stops at line 20. Examples: Python, BASIC, JavaScript in a browser, Unix shell.

8.5 Compiler versus Interpreter

Point of difference	Compiler	Interpreter
Translation unit	Whole program at once	One statement at a time
Output	Produces an object / executable file	No object file; executes directly
Execution speed	Fast: translation is done only once	Slower: translation happens every time the program runs
Translation time	Longer (whole program)	Shorter per statement
Error reporting	All syntax errors reported together after scanning the whole program	Stops at the first error in the statement being executed
Memory	Needs more memory during compilation	Needs less memory
Distribution	Executable can be given to users without the source code	Source code must be given along with the interpreter
Debugging	Harder: errors are reported after compiling	Easier: errors shown at the exact line as it runs
Languages	C, C++, Go, Rust, FORTRAN	Python, BASIC, JavaScript, Ruby, shell scripts

Some languages use **both**. Java first compiles source code into intermediate *bytecode*, which is then interpreted (and just-in-time compiled) by the Java Virtual Machine. Python also compiles to bytecode internally before interpreting it.

Key points

- Assembler: assembly → machine code. Compiler: whole high-level program → machine code. Interpreter: line by line.
- Compiled programs run faster; interpreted programs are easier to debug.
- C is a compiled language; GCC is the most common C compiler.

9. Loading and Linking of a Program

Compiling alone does not give you a program that can run. A C program uses library functions such as **printf** and **scanf** whose machine code was written by someone else and lives in library files. Two more system programs, the **linker** and the **loader**, complete the journey from source code to a running program.

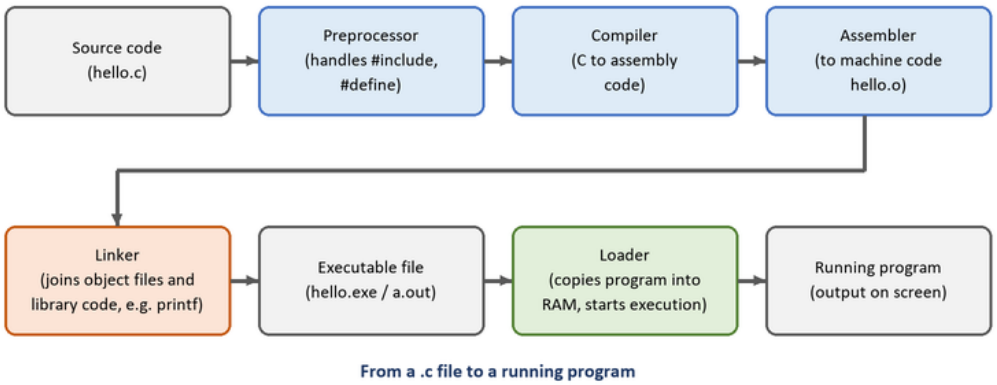


Figure 9.1 – Stages in converting a C source file into a running program

9.1 The Complete Translation Chain

1. **Editor** – the programmer types the source code and saves it as a text file with the extension **.c** (for example **hello.c**).
2. **Preprocessor** – runs before the compiler. It processes the lines that begin with **#**: **#include** copies the contents of a header file (such as **stdio.h**) into the program, **#define** replaces symbolic constants with their values, and comments are removed. The output is an expanded source file (**hello.i**).
3. **Compiler** – translates the expanded C code into assembly code (**hello.s**) after checking syntax and semantics.
4. **Assembler** – converts the assembly code into machine code and stores it in an **object file** (**hello.o** on Linux, **hello.obj** on Windows). This file is not yet runnable because calls to library functions are still unresolved.
5. **Linker** – combines the object file with the object code of the library functions it uses (and with any other object files of the project) to produce a single **executable file** (**a.out** on Linux, **hello.exe** on Windows).
6. **Loader** – part of the operating system. When you run the program, the loader copies the executable from the disk into RAM, sets up the memory areas it needs, places the starting address in the program counter and hands control to the CPU.

9.2 Linking

Linking is the process of joining together several pieces of object code into one executable. The linker resolves *external references*: when your code calls **printf**, the compiler leaves a note saying "address of printf to be filled in later", and the linker fills in the real address after finding **printf** in the standard C library.

Static linking	Dynamic linking
Library code is copied **into** the executable at link time.	Only a reference to the library is stored; the library (DLL on Windows, .so on Linux) is loaded into memory at run time and shared.
Executable is larger but self-contained.	Executable is smaller; several programs share one copy of the library in memory.

Static linking	Dynamic linking
Faster start-up; no dependency problems.	Library can be updated without recompiling the program; program fails if the library file is missing.

9.3 Loading

Loading is the process of placing the executable program into main memory so that it can be executed. The loader also performs **relocation** (adjusting addresses inside the program to match where it was actually placed in memory) and then starts execution. Loading is invisible to the programmer; it happens automatically when you double-click an **.exe** file or type **./a.out**.

9.4 Compiling a C Program with GCC

On Linux, macOS or Windows (with MinGW or Code::Blocks installed), a C program is compiled and run from the command line as shown. The **-o** option names the output file; without it GCC produces **a.out** (Linux) or **a.exe** (Windows).

```
gcc hello.c -o hello       (preprocess + compile + assemble + link, all in one command)
./hello                    (Linux / macOS: the loader runs the program)
hello.exe                  (Windows)
```

Key points

- Chain: Editor → Preprocessor → Compiler → Assembler → Linker → Loader → Execution.
- Preprocessor handles # lines; compiler produces object code; linker adds library code; loader puts the program in RAM.
- Object file (.o / .obj) is not executable; the linker produces the executable (.exe / a.out).

10. Developing a Program

Writing a program is not just typing code. A good programmer follows a systematic set of steps so that the program is correct, easy to understand and easy to change later. These steps form the **program development life cycle**.

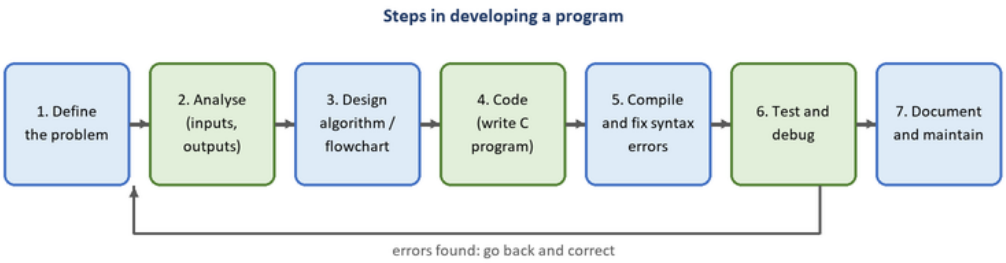


Figure 10.1 – Steps in developing a program

10.1 Steps in Program Development

1. **Problem definition** – understand exactly what the program must do. Read the problem statement carefully; ask what is given and what is required. Example: "Compute the average marks of a student from marks in five subjects."

2. **Problem analysis** – identify the **inputs** (five marks), the **outputs** (the average) and the **processing** (add the five marks and divide by 5). Decide what data types are needed and what special cases exist (negative marks are invalid).
3. **Algorithm design** – write the solution as an ordered sequence of steps in plain English (algorithm), as pseudo-code or as a flowchart. This is done **before** coding.
4. **Coding** – translate the algorithm into a programming language such as C using an editor or IDE.
5. **Compilation** – convert the source code into machine code. The compiler reports **syntax errors**; fix them and compile again until there are none.
6. **Testing and debugging** – run the program with different sets of input, including unusual values, and compare the actual output with the expected output. Finding and removing errors is called **debugging**.
7. **Documentation** – write comments inside the program and a short user manual describing how to use it, so that others (and you, six months later) can understand it.
8. **Maintenance** – after the program is in use, correct any new errors, adapt it to new requirements and improve it.

10.2 Types of Errors in a Program

Type of error	When it is found	Cause	Example
Syntax error	At compile time	Violation of the grammar rules of the language	Missing semicolon: **int a = 5** (no ;)
Semantic / type error	At compile time	Statement is grammatical but meaningless	Assigning a string to an int variable
Linker error	At link time	A function is used but its definition is not found	Calling **Printf** instead of **printf**
Run-time error	While the program runs	An operation that cannot be performed	Division by zero, accessing memory outside an array
Logical error	Only by testing	Program runs but gives wrong output because the logic is wrong	Using **a * b** when **a + b** was needed; the compiler cannot detect this

Testing shows that errors exist; **debugging** finds where they are and removes them. A program that compiles without errors is not necessarily correct; logical errors are the hardest to find and are caught only by careful testing with well-chosen inputs.

10.3 Qualities of a Good Program

- **Correctness** – produces the right output for all valid inputs.
- **Readability** – meaningful variable names, indentation and comments so that others can follow it.
- **Efficiency** – uses CPU time and memory sensibly.
- **Modularity** – divided into small functions, each doing one job.
- **Robustness** – handles invalid input gracefully instead of crashing.
- **Portability** – runs on different machines with little or no change.

Key points

- Steps: define → analyse → design (algorithm/flowchart) → code → compile → test and debug → document → maintain.
- Syntax errors are caught by the compiler; logical errors are caught only by testing.
- Debugging = locating and removing errors.

11. Software Development

11.1 What is Software Development?

A single program written by one student is small enough to be built with the steps in Chapter 10. Real software, such as a banking system or a mobile app, involves thousands of programs, many programmers and years of work. **Software development** (also called software engineering) is the disciplined process of building such large software so that it is delivered on time, within budget and with the required quality.

11.2 The Software Development Life Cycle (SDLC)

The SDLC is the sequence of phases through which software passes from the first idea to retirement.

Phase	What happens	Output
1. Requirement analysis	Meet the customer, understand what the software must do and the constraints (cost, time, hardware). Requirements are written down clearly.	Software Requirements Specification (SRS)
2. Feasibility study	Check whether the project is possible technically, economically and operationally.	Feasibility report
3. Design	Decide the overall structure: modules, data, files, user interface, algorithms. High-level design first, then detailed design.	Design document, flowcharts, data-flow diagrams
4. Coding (implementation)	Programmers write the programs for each module in the chosen language.	Source code
5. Testing	Each module is tested (unit testing), then modules are combined and tested together (integration testing), then the whole system is tested against the requirements (system and acceptance testing).	Test reports, bug fixes
6. Deployment (installation)	The software is installed on the customer's computers and users are trained.	Working system
7. Maintenance	Errors found after release are corrected, and the software is updated as needs change. Usually the longest and costliest phase.	New versions

11.3 Software Development Models

- **Waterfall model** – the phases are carried out strictly one after another, like water flowing down steps; a phase begins only when the previous one is complete. Simple and easy to manage, but changes are difficult once a phase is over. Suitable when requirements are clear and fixed.
- **Iterative / incremental model** – the software is built in small pieces (increments); each increment goes through analysis, design, coding and testing, and adds working features. Customers see results early.

- **Spiral model** – combines iteration with risk analysis; each loop of the spiral is one version of the software.
- **Agile model** – development in short cycles called *sprints* (two to four weeks), with constant customer feedback and frequent releases. Widely used in industry today (Scrum, Kanban).

11.4 Characteristics of Good Software

Characteristic	Meaning
Functionality	Does what the user needs, correctly.
Reliability	Works without failure for long periods.
Usability	Easy to learn and use; clear interface.
Efficiency	Uses CPU, memory and network sensibly.
Maintainability	Easy to correct, improve and adapt.
Portability	Can be moved to a different hardware or OS with little effort.
Security	Protects data from unauthorised access.

Key points

- SDLC phases: Requirements → Feasibility → Design → Coding → Testing → Deployment → Maintenance.
- Waterfall = strictly sequential; Agile = short iterative cycles with feedback.
- Maintenance is usually the longest and most expensive phase.

PART C | ALGORITHMS AND STRUCTURED PROGRAMMING

12. Algorithms

12.1 Definition

An **algorithm** is a finite sequence of well-defined, unambiguous steps that, when followed, solve a specific problem or perform a specific task. The word comes from the name of the Persian mathematician Al-Khwarizmi. An algorithm is written in plain language, independent of any programming language, and is the **plan** that is later converted into a program.

A recipe is an everyday algorithm: it lists ingredients (input), gives numbered steps (processing) and ends with the dish (output). The steps must be clear enough that anyone can follow them and get the same result.

12.2 Characteristics of a Good Algorithm

Donald Knuth listed five properties that every algorithm must have:

Property	Meaning
Input	Zero or more quantities are supplied from outside before the algorithm begins.
Output	At least one quantity is produced as the result.
Definiteness	Every step must be precisely and unambiguously defined. "Add a little salt" is not definite; "add 5 grams of salt" is.
Finiteness	The algorithm must terminate after a finite number of steps. It must not run forever.

Property	Meaning
Effectiveness	Every step must be basic enough to be carried out exactly, in a finite amount of time, by a person with pen and paper.

In addition, a good algorithm should be **correct** (gives the right answer for all valid inputs), **efficient** (uses as little time and memory as possible) and **general** (works for a class of problems, not just one particular input).

12.3 Why Write an Algorithm before Coding?

- It forces you to think about the logic **before** worrying about the syntax of a language.
- Errors in logic are cheaper to fix on paper than in code.
- It is language independent: the same algorithm can be coded in C, Java or Python.
- It is a form of documentation that others can read without knowing the language.

12.4 Examples of Simple Algorithms

Example 1: Find the sum and average of three numbers

```
Step 1: Start
Step 2: Read three numbers a, b, c
Step 3: sum = a + b + c
Step 4: average = sum / 3
Step 5: Print sum and average
Step 6: Stop
```

Example 2: Find whether a number is even or odd

```
Step 1: Start
Step 2: Read a number n
Step 3: Compute remainder r = n mod 2
Step 4: If r = 0 then print "Even"
        otherwise print "Odd"
Step 5: Stop
```

Example 3: Find the largest of three numbers

```
Step 1: Start
Step 2: Read a, b, c
Step 3: If a > b and a > c then largest = a
Step 4: Else if b > c then largest = b
Step 5: Else largest = c
Step 6: Print largest
Step 7: Stop
```

Example 4: Find the factorial of a number n (n! = 1 × 2 × 3 × … × n)

```
Step 1: Start
Step 2: Read n
Step 3: fact = 1, i = 1
Step 4: Repeat while i <= n
        fact = fact * i
        i = i + 1
Step 5: Print fact
Step 6: Stop
```

Key points

- Algorithm = finite, definite, effective sequence of steps with input and output.
- Knuth's five properties: Input, Output, Definiteness, Finiteness, Effectiveness.
- Algorithm first, code later.

13. Different Ways of Stating Algorithms

The same algorithm can be expressed in three common forms: as **step-form** (numbered English sentences), as **pseudo-code** (structured English that looks like a program) and as a **flowchart** (a diagram). Each has its place; in an examination you may be asked to write any of them for a given problem.

13.1 Step-form (Natural-language steps)

The algorithm is written as a numbered list of plain-English steps, as in the examples of Chapter 12. It is the easiest form for beginners and needs no special notation. Its weakness is that English can be ambiguous and long algorithms with many decisions and loops become hard to follow.

13.2 Pseudo-code

Pseudo-code ("false code") is a compact, informal way of describing an algorithm using the structure and keywords of a programming language (IF, ELSE, WHILE, FOR, READ, PRINT) combined with plain English. It is not meant to be compiled; it is meant to be read by humans and then translated easily into any real language.

Common pseudo-code keywords

Purpose	Keywords used
Start and end	BEGIN … END
Input and output	READ, INPUT, GET / PRINT, WRITE, DISPLAY
Assignment	SET x TO 5, x ← 5, x = 5
Selection (decision)	IF condition THEN … ELSE … ENDIF
Repetition (loop)	WHILE condition DO … ENDWHILE; FOR i = 1 TO n DO … ENDFOR; REPEAT … UNTIL condition

Example: Pseudo-code for the sum of the first N natural numbers

```
BEGIN
   READ N
   SET sum TO 0
   SET i TO 1
   WHILE i <= N DO
      sum = sum + i
      i = i + 1
   ENDWHILE
   PRINT sum
END
```

Example: Pseudo-code to check whether a number is prime

```
BEGIN
   READ n
   SET isPrime TO true
   FOR i = 2 TO n - 1 DO
      IF n mod i = 0 THEN
         SET isPrime TO false
      ENDIF
   ENDFOR
   IF isPrime = true THEN PRINT "Prime"
   ELSE PRINT "Not prime"
   ENDIF
END
```

Advantages of pseudo-code	Disadvantages of pseudo-code
Language independent; easy to convert to any programming language.	No standard format; different people write it differently.
Shows the structure (loops, decisions) clearly using indentation.	Cannot be compiled or executed, so errors are not detected automatically.
Faster to write and modify than a flowchart.	Less visual than a flowchart for beginners.

13.3 Flowchart

A **flowchart** is a pictorial representation of an algorithm. Each step is drawn inside a standard symbol and the symbols are connected by arrows (flow lines) that show the order in which the steps are carried out. Flowcharts were popularised by IBM in the 1950s and their symbols are standardised by ANSI/ISO.

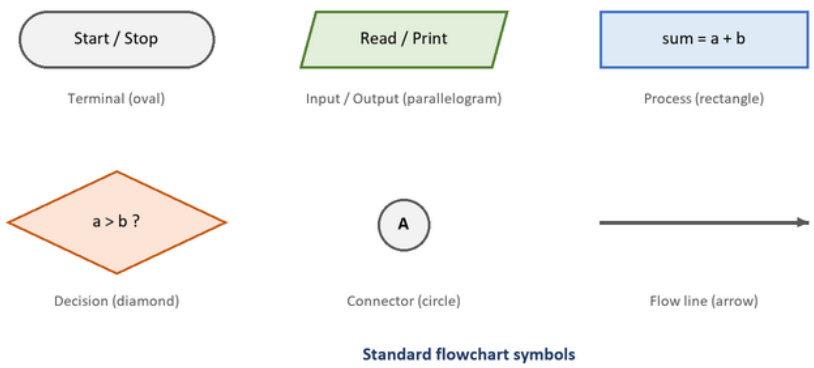


Figure 13.1 – Standard flowchart symbols

Symbol	Name	Used for
Oval / rounded rectangle	Terminal	Start and Stop of the flowchart. Every flowchart has exactly one Start.
Parallelogram	Input / Output	Reading data (Read, Input) or displaying results (Print, Display).
Rectangle	Process	Calculations and assignments, e.g. sum = a + b.
Diamond	Decision	A question with a Yes/No (True/False) answer; two exits.
Circle	Connector	Joins two parts of a flowchart on the same page (used when lines would cross).
Pentagon (home-plate shape)	Off-page connector	Continues the flowchart on another page.
Arrow	Flow line	Shows the direction of flow from one symbol to the next.
Rectangle with double side lines	Pre-defined process	A call to a subroutine / function defined elsewhere.

Rules for drawing a flowchart

- Every flowchart begins with a Start symbol and ends with a Stop symbol.
- Flow is normally from top to bottom and left to right.
- Only one flow line enters and leaves a process symbol; a decision symbol has one entry and two exits, labelled Yes/No or True/False.
- Flow lines should not cross; use connectors instead.
- Use short, clear statements inside the symbols; use standard symbols only.

Example 1: Largest of two numbers (decision)

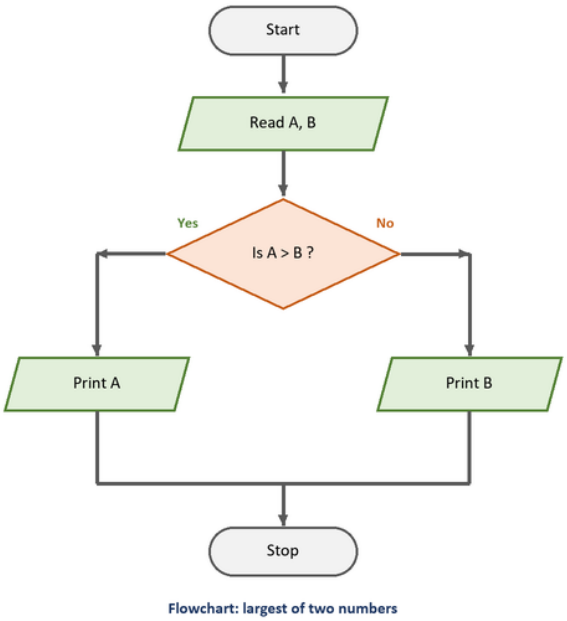


Figure 13.2 – Flowchart to find the larger of two numbers

Example 2: Sum of the first N natural numbers (loop)

This flowchart introduces a **loop**. The variable **i** acts as a counter. The decision "Is i <= N?" is tested each time round; as long as it is true the body (add **i** to **sum**, increase **i**) is repeated. When **i** becomes greater than **N** the loop ends and the sum is printed. Notice that **sum** and **i** are **initialised** before the loop begins; forgetting this is a common mistake.

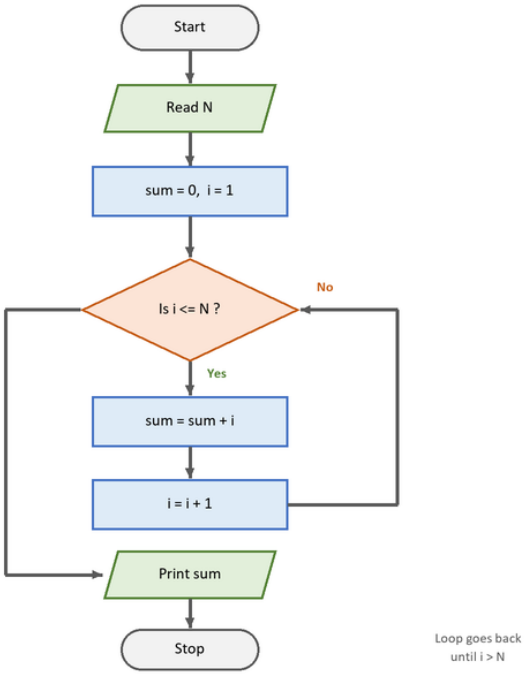


Figure 13.3 – Flowchart to find the sum 1 + 2 + … + N

Advantages of flowcharts	Disadvantages of flowcharts
Easy to understand at a glance, even for non-programmers (communication).	Time-consuming to draw and redraw when the logic changes.
Helps in analysing the problem and finding logical errors before coding.	Become very large and complex for big programs.

Advantages of flowcharts	Disadvantages of flowcharts
Acts as a guide while coding and as documentation afterwards.	No standard for how much detail to show.
Makes debugging and maintenance easier.	Cannot be typed easily; needs drawing tools.

13.4 Comparison of the Three Forms

Aspect	Step-form	Pseudo-code	Flowchart
Form	Numbered English sentences	Structured English with keywords	Diagram with standard symbols
Best for	Very simple problems, beginners	Problems with loops and decisions; close to code	Visualising the flow of control
Ease of change	Easy	Easy	Difficult (redraw)
Closeness to a program	Low	High	Medium

Key points

- Three ways: step-form, pseudo-code, flowchart.
- Flowchart symbols: oval (start/stop), parallelogram (I/O), rectangle (process), diamond (decision), circle (connector), arrow (flow).
- A decision has two exits; a loop is a decision whose one exit goes back to an earlier step.

14. Strategy for Designing Algorithms

14.1 The Problem-Solving Approach

Beginners often start typing code the moment they read a problem. Experienced programmers first **understand**, then **plan**, then **code**. The mathematician George Pólya described problem solving in four stages, which apply directly to programming:

1. **Understand the problem** – What is given (input)? What is required (output)? What are the conditions and constraints? Work out a small example by hand.
2. **Devise a plan** – Break the problem into smaller sub-problems. Have you seen a similar problem before? Which technique fits: a formula, a search, a loop, a decision?
3. **Carry out the plan** – Write the algorithm (step-form, pseudo-code or flowchart) and then the program.
4. **Look back** – Test with several inputs. Can the solution be made simpler or faster? Does it handle special cases (zero, negative numbers, empty input)?

14.2 Top-down Design (Stepwise Refinement)

In **top-down design** the whole problem is first stated as a single high-level step, then broken into a few sub-steps, and each sub-step is broken down further until every step is simple enough to code directly. This gradual breaking-down is called **stepwise refinement**. It is the strategy used with structured programming and with C functions: **main()** calls a few functions, each of which calls smaller functions.

Example: "Prepare a student's mark sheet" refines into (1) read the student's details and marks, (2) compute total and percentage, (3) decide the grade, (4) print the mark sheet. Step (3) refines further into "if percentage ≥ 90 grade A, else if ≥ 75 grade B, …".

14.3 Bottom-up Design

In **bottom-up design** the small, basic components are built and tested first, and are then combined into larger components until the whole system is complete. This approach suits situations where reusable modules or library functions already exist. In practice most projects use a mixture of both approaches.

Top-down design	Bottom-up design
Starts from the overall problem and breaks it into modules.	Starts from the smallest modules and combines them.
Good for understanding the complete structure early.	Good when tested building blocks already exist (reuse).
Used in structured / procedural programming (C).	Common in object-oriented programming (C++, Java).
Testing of lower modules is delayed.	Overall integration is delayed.

14.4 Common Algorithm-design Techniques

Technique	Idea	Example
Brute force	Try all possibilities and pick the one that works.	Checking every number from 2 to n−1 to test whether n is prime
Divide and conquer	Split the problem into smaller independent parts of the same kind, solve each, combine the answers.	Binary search, merge sort
Greedy	At each step choose the option that looks best right now.	Giving change with the fewest coins
Dynamic programming	Solve overlapping sub-problems once and remember the results.	Fibonacci numbers, shortest paths
Backtracking	Build a solution step by step; undo a step when it leads nowhere.	Solving Sudoku, the 8-queens puzzle

In Unit 1 you only need to know the names and basic idea of these techniques; detailed study comes in the Data Structures and Algorithms course.

14.5 Efficiency of an Algorithm

Two algorithms may both be correct yet one may be far faster. The **time complexity** of an algorithm describes how the running time grows as the input size *n* grows, and **space complexity** describes memory usage. For example, to find the sum 1 + 2 + … + n, a loop takes about *n* steps, while the formula *n(n+1)/2* takes one step regardless of *n*. Choosing a better algorithm usually gives a bigger speed-up than buying a faster computer.

Key points

- Understand → Plan → Code → Test (Pólya's four steps).
- Top-down design = stepwise refinement from the whole problem to small steps; bottom-up = build small parts first.
- Techniques: brute force, divide and conquer, greedy, dynamic programming, backtracking.
- A better algorithm beats a faster machine.

15. Structured Programming Concept

15.1 What is Structured Programming?

Structured programming is a programming approach in which a program is built using only three basic control structures (**sequence**, **selection** and **iteration**), organised into small, well-defined **modules** (functions), with a single entry and single exit for each block. The approach was introduced in the late 1960s by Edsger Dijkstra and others; his famous 1968 letter "Go To Statement Considered Harmful" argued against the uncontrolled jumps that made older programs ("spaghetti code") impossible to follow. C is designed as a structured programming language.

15.2 The Three Control Structures

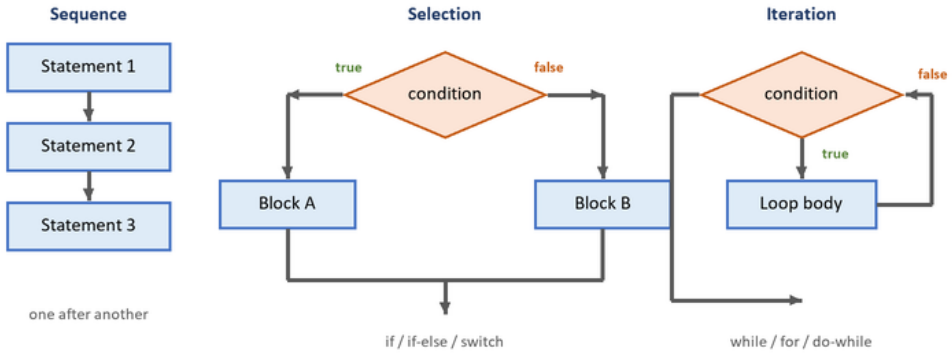


Figure 15.1 – Sequence, selection and iteration: every program can be written using only these three

Structure	Meaning	C statements
Sequence	Statements are executed one after another in the order written.	Any list of statements ending with ;
Selection (decision, branching)	A condition is tested and one of two or more paths is chosen.	**if**, **if-else**, **else-if** ladder, **switch**
Iteration (repetition, looping)	A block of statements is repeated as long as a condition remains true.	**while**, **for**, **do-while**

The **structured program theorem** (Böhm and Jacopini, 1966) proves that any computable task can be programmed using only these three structures. Therefore the **goto** statement, which jumps to an arbitrary label, is never necessary and is avoided in structured programming.

15.3 Modular Programming

A large program is divided into smaller, independent, self-contained parts called **modules** (in C, **functions**). Each module performs one clearly defined task, has a well-defined interface (inputs and outputs), and can be written and tested separately. **main()** acts as the controller that calls the other modules in the right order. Modules are the natural result of top-down design.

15.4 Advantages of Structured Programming

- **Readability** – the flow of control is clear because there are no arbitrary jumps.
- **Easier debugging and testing** – each module can be tested on its own; errors are localised.
- **Easier maintenance** – changes in one module do not affect others.
- **Reusability** – a module written once (e.g. a function to find a factorial) can be used in many programs.
- **Team work** – different programmers can write different modules at the same time.

- **Reduced complexity** – top-down design and small modules keep each part simple.

Disadvantages

- Data and functions are separate; data is not protected from being changed by any function (solved later by object-oriented programming).
- Not ideal for very large systems where objects model the problem better.

15.5 Structured versus Unstructured Programming

Structured programming	Unstructured programming
Uses sequence, selection and iteration only; no goto.	Uses goto and arbitrary jumps freely.
Program divided into modules / functions.	Program is one long block of code.
Easy to read, test, debug and maintain.	Hard to follow ("spaghetti code"), hard to modify.
Languages: C, Pascal, ALGOL.	Early BASIC, assembly language, early FORTRAN.

Key points

- Three control structures: sequence, selection (if/switch), iteration (while/for/do-while).
- Every block has one entry and one exit; goto is avoided.
- Modular = program divided into functions, each doing one job.
- Dijkstra (1968): "Go To Statement Considered Harmful".

PART D | BASICS OF C

16. Overview of C

16.1 History of C

C was developed by **Dennis Ritchie** at **Bell Laboratories (AT&T), USA, in 1972**. It grew out of an earlier language called **B**, written by Ken Thompson, which in turn was derived from **BCPL** (Basic Combined Programming Language). Ritchie created C to rewrite the **UNIX operating system**, which until then had been written in assembly language. Because C was portable and efficient, UNIX could then be moved easily to new computers, and the language spread with it.

Year	Milestone
1960s	ALGOL, then BCPL (Martin Richards, 1967) and B (Ken Thompson, 1970)
1972	C created by Dennis Ritchie at Bell Labs
1978	The book "The C Programming Language" by Kernighan and Ritchie (K&R C) becomes the informal standard
1989 / 1990	ANSI C (C89) and ISO C (C90) – the first official standard
1999	C99 – adds **//** comments, **long long**, **bool**, variable-length arrays
2011 / 2018	C11 and C17 – multithreading support, minor corrections
2024	C23 – the latest standard

16.2 Features of C

- **Simple and small** – only 32 keywords (in C89) and a small set of built-in operations; everything else comes from libraries.
- **Structured language** – programs are built from functions and the three control structures.
- **Middle-level language** – combines high-level features (functions, data types) with low-level access (pointers, bitwise operators, direct memory manipulation).
- **Portable** – a C program written on one machine can be compiled and run on another with little or no change.
- **Fast and efficient** – compiled to machine code; very close to the speed of assembly language.
- **Rich library** – standard functions for input/output, string handling, mathematics, memory management (**stdio.h**, **string.h**, **math.h**, **stdlib.h**).
- **Extensible** – programmers can write their own functions and add them to libraries.
- **Case sensitive** – **sum**, **Sum** and **SUM** are three different names.
- **Pointers** – allow direct interaction with memory addresses; the basis for dynamic memory and data structures.

16.3 Uses of C

Operating systems (UNIX, Linux, the Windows kernel), compilers and interpreters (the Python interpreter is written in C), embedded systems (microcontrollers in cars, washing machines, medical devices), database systems (MySQL, PostgreSQL), device drivers, network software, game engines and system utilities. C is also the first language taught in most engineering courses because it teaches how the computer actually works.

16.4 Advantages and Disadvantages

Advantages	Disadvantages
Fast execution; small memory footprint.	No built-in object-oriented features (no classes, inheritance).
Portable across platforms.	No automatic memory management; the programmer must free memory.
Foundation for C++, Java, C#, JavaScript syntax.	No run-time checking of array bounds; mistakes can crash the program.
Direct hardware and memory access through pointers.	Pointers are powerful but error-prone for beginners.
Large community, many libraries and tools.	No built-in exception handling or string data type.

Key points

- C: Dennis Ritchie, Bell Labs, 1972, to write UNIX. Derived from B and BCPL.
- Standards: K&R (1978) → ANSI C89/C90 → C99 → C11 → C17 → C23.
- Structured, middle-level, portable, compiled, case-sensitive language with 32 keywords.

17. Developing Programs in C

17.1 What You Need

- **A text editor** to type the program: Notepad++, VS Code, gedit, or the editor built into an IDE.

- **A C compiler** to translate it: GCC (part of MinGW on Windows, pre-installed on most Linux systems), Clang, or Microsoft Visual C++.
- Optionally an **IDE (Integrated Development Environment)** that combines editor, compiler, debugger and output window in one program: Code::Blocks, Dev-C++, Visual Studio Code with the C/C++ extension, Turbo C (old but still used in some labs), or online compilers such as OnlineGDB and Programiz.

17.2 Steps to Write, Compile and Run a C Program

1. **Write** the program in the editor and **save** it with the **.c** extension, for example **first.c**. The file name is the *source file*.
2. **Compile** it. In an IDE press the Build/Compile button; on the command line type **gcc first.c -o first**. The compiler checks the syntax. If there are errors it prints the line number and a message; correct them and compile again.
3. **Link** happens automatically with the compile command; the executable **first.exe** (Windows) or **first** (Linux) is created.
4. **Run** the executable: press Run in the IDE, or type **first** / **./first** on the command line. The loader places the program in memory and it starts executing from **main()**.
5. **Check the output**. If it is wrong, the error is logical: go back to the editor, correct the program, and repeat from step 2.

17.3 Files Created during Compilation

File	Extension (Windows / Linux)	Created by	Contains
Source file	.c / .c	Programmer (editor)	C program text
Header file	.h / .h	Library or programmer	Declarations of functions and constants (e.g. stdio.h)
Object file	.obj / .o	Compiler	Machine code of this file, library calls not yet resolved
Executable file	.exe / (no extension, a.out)	Linker	Complete runnable machine code

17.4 Your First C Program

Program 17.1 – hello.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Hello, World!\n");
6      return 0;
7  }
```

Output

```
Hello, World!
```

Line-by-line explanation

Line	Code	Explanation
1	#include <stdio.h>	A preprocessor directive. It tells the preprocessor to include the **standard input/output header file** before compilation. This file contains the declaration of **printf()** and **scanf()**. Without it the compiler does not know what **printf** is.
2	(blank line)	Blank lines are ignored by the compiler; they improve readability.
3	int main()	Every C program must have exactly one function named **main**; execution starts here. **int** says that main returns an integer value to the operating system. The empty brackets mean it takes no arguments.
4	{	Opening brace: the beginning of the body of **main**.
5	printf("Hello, World!\n");	Calls the library function **printf** to display the text between the double quotes on the screen. **\n** is the newline character, which moves the cursor to the next line. The semicolon ends the statement.
6	return 0;	Sends the value 0 back to the operating system, meaning "program finished successfully".
7	}	Closing brace: the end of **main**. The program stops here.

Key points

- Save as .c → compile (gcc file.c -o file) → run. Fix syntax errors at compile time, logical errors after testing.
- Every C program needs #include <stdio.h> for printf/scanf and exactly one main() function.
- Statements end with a semicolon; the body of a function is enclosed in { }.

18. Parts and Structure of a C Program

18.1 Parts of a Simple C Program

A simple C program, such as the one in Chapter 17, has the following parts: **preprocessor directives** (lines beginning with **#**), the **main() function** with its **header** (**int main()**) and its **body** (everything between **{** and **}**), **declarations** of variables inside the body, **statements** that do the work, and a **return statement**. Larger programs add comments, symbolic constants, global variables and user-defined functions. Together these make up the general **structure of a C program**.

18.2 General Structure of a C Program

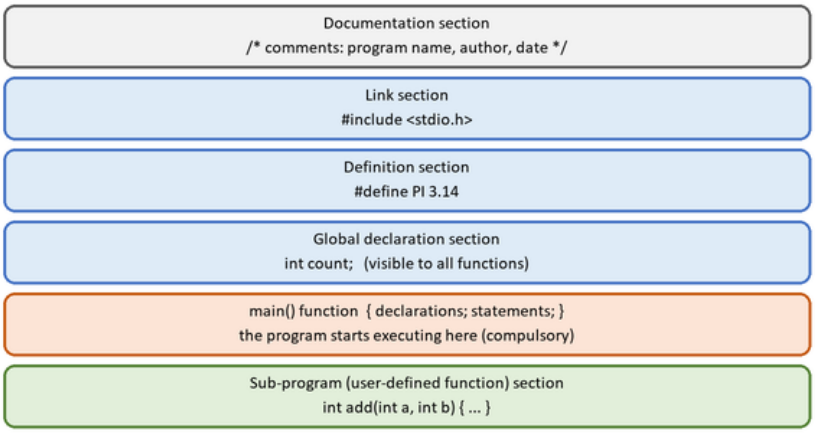


Figure 18.1 – The six sections of a C program, in the order they normally appear

Section	Purpose	Compulsory?
Documentation section	Comments giving the program name, author, date and a short description. Ignored by the compiler.	No
Link (preprocessor) section	**#include** directives that bring in header files such as **<stdio.h>**, **<math.h>**.	No (but practically always present)
Definition section	**#define** directives that define symbolic constants, e.g. **#define PI 3.14159**.	No
Global declaration section	Variables (and function prototypes) declared outside all functions; they are visible to every function in the program.	No
main() function section	The heart of the program. Contains a declaration part (variables) followed by an executable part (statements). Execution starts and ends here.	Yes
Sub-program section	User-defined functions called from **main()** or from each other. Usually placed after **main()**, with prototypes declared above.	No

18.3 A Program Showing All the Sections

Program 18.1 – area.c (all six sections)

```
1  /* Program  : area.c
2     Purpose  : Find the area of a circle
3     Author   : CSE Department        Date : 05-09-2026 */
4
5  #include <stdio.h>            /* link section */
6
7  #define PI 3.14159            /* definition section */
8
9  float radius;                 /* global declaration section */
10
11 float area(float r);           /* function prototype */
12
13 int main()                    /* main() function section */
14 {
15    float a;                   /* local declaration */
16    printf("Enter the radius: ");
17    scanf("%f", &radius);
18    a = area(radius);
19    printf("Area of the circle = %.2f\n", a);
20    return 0;
21 }
22
23 float area(float r)           /* sub-program section */
24 {
25    return PI * r * r;
26 }
```

Output

```
Enter the radius: 2
Area of the circle = 12.57
```

Explanation

- **Lines 1–3** are one multi-line comment: the documentation section.
- **Line 5** includes **stdio.h** so that **printf** and **scanf** can be used.
- **Line 7** defines the symbolic constant **PI**. Wherever **PI** appears, the preprocessor replaces it with 3.14159 before compilation.
- **Line 9** declares a global variable **radius** that can be used inside any function.
- **Line 11** is a **function prototype**: it tells the compiler that a function named **area** exists, takes one **float** and returns a **float**. This allows **main()** to call **area()** even though **area()** is written later.
- **Lines 13–21** are **main()**. **float a;** is a local variable; **scanf("%f", &radius)** reads a real number from the keyboard into **radius** (the **&** gives the address of the variable); **a = area(radius);** calls the function; **%.2f** prints the result with two decimal places.
- **Lines 23–26** define the function **area()**, which computes and returns πr^2.

18.4 The main() Function

- Every C program must contain exactly one **main()**; execution begins with the first statement of **main()** and ends when **main()** returns.

- The standard forms are **int main(void)** and **int main(int argc, char *argv[])**. Older books write **void main()**, but this is non-standard and should be avoided; always write **int main()** and end with **return 0;**.
- **main()** can call library functions and user-defined functions, but no function can be defined inside another function.

18.5 Header Files

Header file	Provides
stdio.h	Standard input/output: printf, scanf, getchar, putchar, gets, puts, file functions
conio.h	Console I/O (Turbo C only, non-standard): clrscr, getch
math.h	Mathematical functions: sqrt, pow, sin, cos, fabs, floor, ceil
string.h	String functions: strlen, strcpy, strcat, strcmp
stdlib.h	General utilities: malloc, free, rand, abs, exit, atoi
ctype.h	Character tests: isdigit, isalpha, toupper, tolower

Key points

- Six sections: Documentation → Link → Definition → Global declaration → main() → Sub-programs. Only main() is compulsory.
- #include brings in header files; #define creates symbolic constants; both are handled by the preprocessor.
- A function prototype declares a function before it is used; the definition gives its body.

19. Comments and Program Statements

19.1 Comments

A **comment** is text inside a program that is meant for human readers and is completely ignored by the compiler. Comments explain what the code does, why it was written that way, who wrote it and when. Good comments make a program easier to understand, debug and maintain. C provides two kinds:

Type	Syntax	Notes
Multi-line (block) comment	/* text … */	Everything between **/*** and ***/** is ignored, even across several lines. Available in all versions of C. Cannot be nested: **/* a /* b */ c */** is an error because the comment ends at the first ***/**.
Single-line comment	// text	Everything from **//** to the end of that line is ignored. Introduced in C99 (borrowed from C++). Most common in modern code.

Program 19.1 – Using comments

```
1  /* This is a block comment.
2     It can span several lines. */
3
4  #include <stdio.h>
5
6  int main()
7  {
8     int marks = 85;        // single-line comment: marks out of 100
9     /* printf("Debug: %d\n", marks); */   /* this line is commented out */
10     printf("Marks = %d\n", marks);
11     return 0;
12  }
```

Output

```
Marks = 85
```

Guidelines for good comments

- Write a header comment at the top of every program: name, purpose, author, date.
- Comment **why** something is done, not the obvious **what** (**i = i + 1; // add 1 to i** is useless).
- Keep comments up to date; a wrong comment is worse than no comment.
- Use comments to temporarily disable code while debugging ("commenting out").

19.2 Program Statements

A **statement** is a complete instruction to the computer. In C most statements end with a **semicolon (;)**, which is called the *statement terminator*. Statements are executed in the order written unless a control statement changes the flow. The main kinds of statements are:

Kind of statement	Purpose	Example
Declaration statement	Creates a variable and tells its type	**int count; float price = 9.5;**
Expression statement	An expression followed by ; (assignment, function call, increment)	**sum = a + b; printf("Hi"); i++;**
Compound statement (block)	Several statements grouped inside { }; treated as one statement	**{ x = 1; y = 2; }**
Selection statement	Chooses between alternatives	**if**, **if-else**, **switch**
Iteration statement	Repeats a block	**while**, **for**, **do-while**
Jump statement	Transfers control	**break**, **continue**, **return**, **goto**
Null statement	Does nothing; just a semicolon	**;**

Important points about statements

- C is a **free-form** language: spaces, tabs and newlines between tokens do not matter. **int a=5;** and **int a = 5 ;** are identical. Use indentation for readability, not because the compiler needs it.
- One statement may be spread over several lines, and several statements may be written on one line (not recommended).
- A **block { … }** does **not** end with a semicolon.
- Preprocessor directives (**#include**, **#define**) are not statements; they do **not** end with a semicolon.

- A missing semicolon is the most common syntax error for beginners; the compiler usually reports it on the **next** line.

Key points

- Comments: /* … */ (multi-line, cannot be nested) and // (single line, C99). Ignored by the compiler.
- Statements end with ; blocks { } and preprocessor directives do not.
- C is free-form and case sensitive.

20. C Tokens, Keywords and Identifiers

20.1 C Tokens

When the compiler reads a program, it first breaks the text into the smallest meaningful units, called **tokens**, just as an English sentence is broken into words and punctuation marks. C has **six types of tokens**:

Token type	Meaning	Examples
Keywords	Reserved words with a fixed meaning	int, float, if, while, return
Identifiers	Names given by the programmer to variables, functions, arrays	sum, main, total_marks, printf
Constants (literals)	Fixed values that do not change	10, 3.14, 'A', 0x1F
Strings	Sequence of characters in double quotes	"Hello", "CSE"
Operators	Symbols that perform operations	+ - * / % = == && <
Special symbols (separators)	Punctuation with special meaning	( ) { } [ ] ; , # . & *

In the statement **int total = marks + 10;** the tokens are: **int** (keyword), **total** (identifier), **=** (operator), **marks** (identifier), **+** (operator), **10** (constant) and **;** (special symbol).

20.2 Keywords

Keywords (reserved words) are words whose meaning is already defined by the C language. They can be used only for their intended purpose and can **never** be used as names for variables, functions or anything else. All keywords are written in **lowercase**. ANSI C (C89) has **32 keywords**:

Table 20.1 – The 32 keywords of ANSI C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Table 20.2 – Keywords grouped by purpose

Group	Keywords
Data types	char, int, float, double, void
Type modifiers	short, long, signed, unsigned
Storage classes	auto, static, extern, register
Type qualifiers	const, volatile
Control flow	if, else, switch, case, default, for, while, do, break, continue, goto, return
User-defined types	struct, union, enum, typedef
Operator	sizeof

C99 added five more (**inline**, **restrict**, **_Bool**, **_Complex**, **_Imaginary**) and C11 added others such as **_Static_assert**, but for Unit 1 the 32 ANSI keywords are what you should remember. Note that **main**, **printf** and **scanf** are **not** keywords; they are identifiers (function names) supplied by the program or the library.

20.3 Identifiers

An **identifier** is the name given by the programmer to a variable, constant, function, array, structure or label. Choosing meaningful identifiers (**total_marks** instead of **t**) is one of the simplest ways to make a program readable.

Rules for forming identifiers

1. An identifier may contain only **letters** (A–Z, a–z), **digits** (0–9) and the **underscore** (_).
2. The **first character must be a letter or an underscore**; it cannot be a digit.
3. **No spaces** or special characters such as **-**, **@**, **$**, **#**, **.** are allowed.
4. A **keyword cannot** be used as an identifier.
5. Identifiers are **case sensitive**: **Marks**, **marks** and **MARKS** are three different identifiers.
6. There is no limit on length in the standard, but only the first **31 characters** are guaranteed to be significant for external names (63 for internal names in C99). Keep names reasonably short.
7. Identifiers beginning with an underscore followed by a capital letter or another underscore are reserved for the compiler and library; avoid them.

Table 20.3 – Valid and invalid identifiers

Identifier	Valid?	Reason
total	Valid	Letters only
total_marks	Valid	Letters and underscore
_count	Valid	May begin with an underscore
marks2	Valid	Digit is allowed after the first character
2marks	Invalid	Cannot begin with a digit
total marks	Invalid	Space not allowed
total-marks	Invalid	Hyphen (minus) not allowed
float	Invalid	float is a keyword
Float	Valid	Case sensitive: Float is not a keyword (but confusing; avoid)
roll.no	Invalid	Dot not allowed
avg$	Invalid	$ not allowed
main	Valid	It is an identifier, but reserved for the main function; do not reuse

Naming conventions (good practice, not rules)

- Variables and functions in lowercase, words separated by underscores or camelCase: **student_age**, **studentAge**.
- Symbolic constants in UPPERCASE: **MAX_SIZE**, **PI**.
- Loop counters are traditionally **i**, **j**, **k**.
- Never use **l** (small L) or **O** alone; they look like 1 and 0.

Key points

- Six tokens: keywords, identifiers, constants, strings, operators, special symbols.
- 32 keywords in ANSI C, all lowercase, cannot be used as names.
- Identifier: letters, digits, underscore; cannot start with a digit; no spaces; case sensitive; not a keyword.

21. Data Types

21.1 What is a Data Type?

A **data type** tells the compiler **what kind of data** a variable holds, **how much memory** to reserve for it and **which operations** are allowed on it. A whole number, a number with a decimal point and a single character are stored differently, so C makes the programmer state the type of every variable before it is used. C is therefore called a **strongly typed** (or statically typed) language.

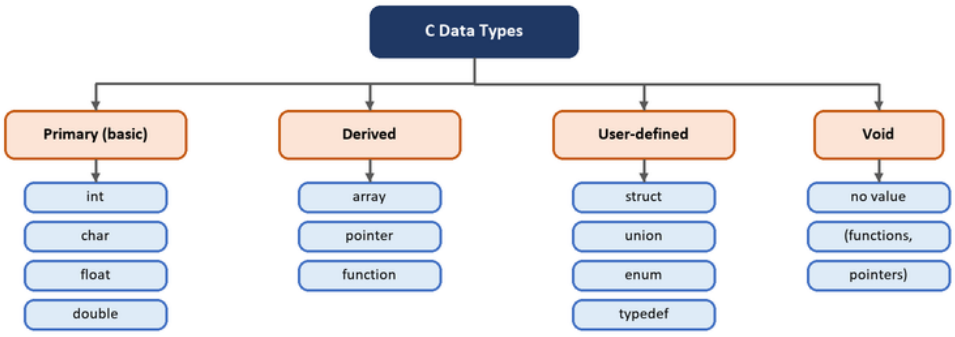


Figure 21.1 – Classification of data types in C

21.2 Primary (Basic / Fundamental) Data Types

The four basic types are **int**, **char**, **float** and **double**, plus **void**. The size of each type depends on the compiler and the machine; the values below are for a typical modern 32-bit / 64-bit compiler such as GCC. (Turbo C, a 16-bit compiler, uses 2 bytes for **int**.)

Table 21.1 – The primary data types (typical 32/64-bit compiler)

Type	Meaning	Size (bytes)	Range	Format specifier
char	A single character (stored as its ASCII code)	1	−128 to 127	%c
int	Whole number (no decimal point)	4	−2,147,483,648 to 2,147,483,647	%d

Type	Meaning	Size (bytes)	Range	Format specifier
float	Real number, single precision (about 6–7 significant digits)	4	$\pm 3.4 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$	%f
double	Real number, double precision (about 15 significant digits)	8	$\pm 1.7 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$	%lf
void	No value / empty type	–	–	–

21.3 Type Modifiers

The modifiers **short**, **long**, **signed** and **unsigned** change the size or the range of the basic integer and character types. **signed** types can hold negative and positive values; **unsigned** types hold only zero and positive values but can go twice as high.

Table 21.2 – Data types with modifiers

Type	Size (bytes)	Range	Format specifier
short int (short)	2	−32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
int (signed int)	4	−2,147,483,648 to 2,147,483,647	%d
unsigned int	4	0 to 4,294,967,295	%u
long int (long)	4 or 8	at least −2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4 or 8	0 to at least 4,294,967,295	%lu
long long int	8	-9.2×10^{18} to 9.2×10^{18}	%lld
signed char	1	−128 to 127	%c
unsigned char	1	0 to 255	%c
long double	8, 12 or 16	larger than double	%Lf

The rule that always holds is: **sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)** and **sizeof(char) = 1**. When the exact size matters, find it with the **sizeof** operator rather than assuming.

21.4 Derived and User-defined Data Types

- **Derived types** are built from the basic types: **arrays** (a collection of elements of the same type, e.g. **int marks[5];**), **pointers** (variables that store memory addresses, e.g. **int *p;**) and **functions** (which return a value of a type).
- **User-defined types** let the programmer create new types: **structure** (**struct**, a group of variables of different types under one name), **union** (**union**, variables sharing the same memory), **enumeration** (**enum**, a list of named integer constants) and **typedef** (gives a new name to an existing type).
- These are studied in detail in later units; in Unit 1 you only need to know the classification.

21.5 The void Type

void means "no value" or "nothing". It is used in three ways: (1) a function that returns nothing is declared **void display(void);**, (2) a function that takes no arguments has **void** in its parameter list, **int main(void)**, and (3) a **void pointer** **void *p;** can point to data of any type. You cannot declare an ordinary variable of type **void**.

21.6 Program: Size of Data Types

The **sizeof** operator gives the number of bytes occupied by a type or a variable. Run this program on your own compiler; the sizes are compiler dependent.

Program 21.1 – sizes.c

```c
#include <stdio.h>

int main()
{
    printf("Size of char       : %zu byte(s)\n", sizeof(char));
    printf("Size of short      : %zu byte(s)\n", sizeof(short));
    printf("Size of int        : %zu byte(s)\n", sizeof(int));
    printf("Size of long       : %zu byte(s)\n", sizeof(long));
    printf("Size of long long  : %zu byte(s)\n", sizeof(long long));
     printf("Size of float      : %zu byte(s)\n", sizeof(float));
     printf("Size of double     : %zu byte(s)\n", sizeof(double));
     printf("Size of long double : %zu byte(s)\n", sizeof(long double));
     return 0;
}
```

Output

```
Size of char       : 1 byte(s)
Size of short      : 2 byte(s)
Size of int        : 4 byte(s)
Size of long       : 8 byte(s)
Size of long long  : 8 byte(s)
Size of float      : 4 byte(s)
Size of double     : 8 byte(s)
Size of long double : 16 byte(s)
```

Explanation: **sizeof(type)** returns a value of type **size_t** (an unsigned integer), so it is printed with **%zu**. On older compilers that do not support **%zu**, cast the value and use **%d**: **printf("%d", (int)sizeof(int));**. The output shown is from GCC on 64-bit Linux; on 64-bit Windows **long** is 4 bytes.

21.7 Program: Using Different Data Types

Program 21.2 – types.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5     int    roll   = 27;
6     char   grade  = 'A';
7     float  cgpa   = 8.75f;
8     double pi     = 3.14159265358979;
9     unsigned int population = 4000000000u;
10
11     printf("Roll number : %d\n", roll);
12     printf("Grade      : %c\n", grade);
13     printf("CGPA       : %.2f\n", cgpa);
14     printf("Pi         : %.10lf\n", pi);
15     printf("Population  : %u\n", population);
16     printf("Grade code  : %d\n", grade);   /* char printed as a number */
17     return 0;
18  }
```

Output

```
Roll number : 27
Grade      : A
CGPA       : 8.75
Pi         : 3.1415926536
Population  : 4000000000
Grade code  : 65
```

Explanation

- **Lines 5–9** declare five variables of different types and initialise them. The suffix **f** in **8.75f** makes the constant a **float** (without it, **8.75** is a **double**); the suffix **u** makes 4000000000 an **unsigned** constant, since it does not fit in a signed **int**.
- **Line 11–15** print each variable with the matching format specifier: **%d** for int, **%c** for char, **%.2f** for float with 2 decimals, **%.10lf** for double with 10 decimals, **%u** for unsigned int.
- **Line 16** prints the same **char** variable with **%d**. A character is stored as its **ASCII code**, so 'A' is printed as 65. This shows that **char** is really a small integer type.

21.8 Common Mistakes with Data Types

- Using the wrong format specifier: printing a **float** with **%d** gives garbage, not an error.
- Storing a value larger than the range: **int x = 3000000000;** overflows and gives a wrong (negative) value.
- Expecting **int** division to give decimals: **7 / 2** is **3**, not **3.5** (see Chapter 24).
- Forgetting that **char** in single quotes ('A') is a character, while "A" in double quotes is a string of two characters ('A' and the terminating '\0').
- Assuming **int** is always 4 bytes; use **sizeof** when it matters.

Key points

- Data type = kind of value + memory size + allowed operations.
- Primary: char (1 byte, %c), int (4 bytes, %d), float (4 bytes, %f), double (8 bytes, %lf), void.
- Modifiers: short, long, signed, unsigned. unsigned doubles the positive range.
- Derived: array, pointer, function. User-defined: struct, union, enum, typedef.
- sizeof gives the size in bytes; sizes are compiler dependent.

22. Variables and Constants

22.1 Variables

A **variable** is a named location in memory that stores a value which can be changed during the execution of the program. Think of it as a labelled box: the **name** (identifier) is the label, the **type** decides the size of the box, and the **value** is what is inside. Every variable in C must be **declared** before it is used.

Declaration

A declaration tells the compiler the name and the type of a variable so that memory can be reserved. Syntax: **data_type variable_name;** or, for several variables of the same type, **data_type name1, name2, name3;**

```
int   count;          /* one integer variable */
float price, discount;   /* two float variables */
char  grade;          /* one character variable */
```

A declared but uninitialised local variable contains a **garbage value** (whatever was in that memory before). Never use a variable before giving it a value.

Initialisation

Giving a variable its first value is called **initialisation**. It can be done at the time of declaration or later with an assignment statement.

```
int count = 0;          /* declaration + initialisation */
float price = 99.50f;
char grade = 'B';
int a = 5, b = 10, c;    /* c is declared but not initialised */
c = a + b;              /* assignment later: c becomes 15 */
```

Rules for variables

- The name must follow the identifier rules (Chapter 20).
- A variable must be declared before use; in C89 all declarations must come at the beginning of a block, in C99 and later they may appear anywhere.
- A variable can be declared only **once** in the same scope, but its value can be changed any number of times.
- The type of a variable cannot change after declaration.

Scope: local and global variables

Local variable	Global variable

Local variable	Global variable
Declared **inside** a function or block.	Declared **outside** all functions, usually at the top of the file.
Can be used only inside that function/block.	Can be used by every function in the program.
Created when the function starts, destroyed when it ends.	Exists for the whole life of the program.
Contains garbage if not initialised.	Automatically initialised to 0 if not initialised.

22.2 Constants

A **constant** (also called a *literal*) is a fixed value that does not change during program execution. **10**, **3.14**, **'A'** and **"Hello"** are constants. C has several kinds:

Type of constant	Description	Examples
Integer constant (decimal)	Whole number; digits 0–9, optional sign; no commas, no decimal point	0, 42, −7, 1000
Integer constant (octal)	Begins with 0; digits 0–7	017 (= 15 decimal), 0777
Integer constant (hexadecimal)	Begins with 0x or 0X; digits 0–9 and A–F	0x1F (= 31 decimal), 0xFF
Integer constant with suffix	u/U = unsigned, l/L = long, ll/LL = long long	100u, 50000L, 9000000000LL
Real (floating-point) constant	Number with a decimal point or exponent; type double by default, f suffix makes it float	3.14, −0.5, 2.0, 1.5e3 (= 1500), 6.02E23, 2.5f
Character constant	A single character in single quotes; value is its ASCII code	'A' (65), 'a' (97), '0' (48), ' ' (32)
String constant	A sequence of characters in double quotes; ends with an invisible null character '\0'	"Hello", "A", "" (empty string)
Escape sequence	A backslash followed by a character; represents a special character	'\n', '\t', '\0', '\\'

Escape sequences

Escape sequence	Meaning	Escape sequence	Meaning
\n	New line	\\	Backslash
\t	Horizontal tab	\'	Single quote
\b	Backspace	\"	Double quote
\r	Carriage return	\?	Question mark
\a	Alert (beep)	\0	Null character (end of string)
\v	Vertical tab	\xhh	Character with hex code hh

22.3 Symbolic Constants: #define and const

If the value 3.14159 is typed in twenty places and later must be changed to 3.14159265, all twenty places must be edited. A **symbolic constant** gives the value a name so that it is written once. C offers two ways:

#define	const
Preprocessor directive: **#define PI 3.14159**	Keyword used in a declaration: **const float PI = 3.14159;**

#define	const
No semicolon, no data type, no =.	Normal declaration with type, = and semicolon.
Simple text replacement before compilation; PI has no type.	PI is a real typed variable whose value cannot be changed.
Not stored in memory; cannot be printed with a debugger.	Occupies memory; visible to the debugger; obeys scope rules.
Conventionally written in UPPERCASE.	Any valid identifier; UPPERCASE by convention.

22.4 Program: Variables and Constants

Program 22.1 – varconst.c

```
1  #include <stdio.h>
2  #define PI 3.14159          /* symbolic constant using #define */
3
4  int main()
5  {
6     const int MAX_MARKS = 100;   /* constant variable using const */
7     int marks = 78;              /* variable, can change */
8     float radius = 2.5f, area;
9     char grade = 'B';
10
11     area = PI * radius * radius;
12     marks = marks + 5;          /* allowed: marks is a variable */
13     /* MAX_MARKS = 90;  ERROR: assignment of read-only variable */
14
15     printf("Marks = %d out of %d\n", marks, MAX_MARKS);
16     printf("Grade = %c (ASCII %d)\n", grade, grade);
17     printf("Area of circle = %.3f\n", area);
18     printf("Hex 0x1F = %d, Octal 017 = %d\n", 0x1F, 017);
19     printf("Tab\tseparated\tvalues\n");
20     return 0;
21 }
```

Output

```
Marks = 83 out of 100
Grade = B (ASCII 66)
Area of circle = 19.635
Hex 0x1F = 31, Octal 017 = 15
Tab    separated      values
```

Explanation

- **Line 2** defines **PI** with **#define**; the preprocessor replaces every **PI** with 3.14159 before compilation.
- **Line 6** declares **MAX_MARKS** with **const**; it behaves like a variable that cannot be assigned to. Line 13 shows that trying to change it is a compile-time error (which is why it is commented out).
- **Line 12** changes the variable **marks** from 78 to 83.
- **Line 16** prints **grade** twice: **%c** shows the character 'B', **%d** shows its ASCII code 66.
- **Line 18** shows that hexadecimal and octal constants are simply other ways of writing integers; **%d** prints their decimal value.
- **Line 19** uses the tab escape sequence **\t** to separate words.

Key points

- Variable = named memory location whose value can change; must be declared with a type before use.
- Uninitialised local variables hold garbage; global variables start at 0.
- Constants: integer (decimal, octal 0…, hex 0x…), real (3.14, 1.5e3), character ('A'), string ("Hi"), escape sequences (\n, \t, \0).
- #define PI 3.14 is text replacement (no ; no type); const float PI = 3.14; is a read-only typed variable.

23. Operators

An **operator** is a symbol that tells the compiler to perform a specific mathematical, logical or other operation on one or more values. The values it works on are called **operands**. In **a + b**, **+** is the operator and **a** and **b** are the operands. Operators are classified by the number of operands: **unary** (one operand, e.g. **-a**, **++i**), **binary** (two operands, e.g. **a * b**) and **ternary** (three operands, only **? :**).

Table 23.1 – Classification of C operators

Category	Operators
Arithmetic	+ − * / %
Relational	< > <= >= == !=
Logical	&& || !
Assignment	= += −= *= /= %= and others
Increment / decrement	++ −−
Bitwise	& | ^ ~ << >>
Conditional (ternary)	? :
Special	sizeof ,(comma) &(address) *(pointer) . −> [ ] ( )

23.1 Arithmetic Operators

Operator	Meaning	Example (a = 17, b = 5)	Result
+	Addition	a + b	22
−	Subtraction	a − b	12
*	Multiplication	a * b	85
/	Division (quotient)	a / b	3 (integer division)
%	Modulus (remainder)	a % b	2

- **Integer division**: when both operands are integers, **/** discards the fractional part. **17 / 5** is **3**, **7 / 2** is **3**, **1 / 2** is **0**. If either operand is a real number the result is real: **17.0 / 5** is **3.4**.
- **Modulus %** gives the remainder and works **only on integers**; **5.5 % 2** is a compile error. The sign of the result follows the sign of the left operand: **-17 % 5** is **-2**, **17 % -5** is **2**.
- **%** is very useful: **n % 2 == 0** tests whether n is even; **n % 10** gives the last digit of n.
- Division by zero (**a / 0** or **a % 0**) is a run-time error; the program crashes.

23.2 Relational Operators

Relational operators compare two values. The result is **1 if the relation is true and 0 if it is false**. They are used in conditions of **if** and loop statements.

Operator	Meaning	Example (a = 10, b = 20)	Result
<	Less than	a < b	1
<=	Less than or equal to	a <= 10	1
>	Greater than	a > b	0
>=	Greater than or equal to	a >= b	0
==	Equal to	a == b	0
!=	Not equal to	a != b	1

Warning: **==** (comparison) and **=** (assignment) are different. **if (a = 5)** assigns 5 to **a** and is always true; **if (a == 5)** tests whether **a** equals 5. This is one of the most common bugs in C.

23.3 Logical Operators

Logical operators combine two or more conditions. In C, **0 is false and any non-zero value is true**; the result of a logical operator is always 0 or 1.

Operator	Meaning	Result
&&	Logical AND	1 only if **both** operands are true. **(a > 5) && (b < 30)**
||	Logical OR	1 if **at least one** operand is true. **(marks < 40) || (attendance < 75)**
!	Logical NOT (unary)	Reverses the truth value: **!0** is 1, **!5** is 0.

Table 23.2 – Truth table for logical operators

A	B	A && B	A || B	!A
0 (false)	0 (false)	0	0	1
0 (false)	1 (true)	0	1	1
1 (true)	0 (false)	0	1	0
1 (true)	1 (true)	1	1	0

Short-circuit evaluation: **&&** and **||** evaluate the left operand first and evaluate the right operand **only if necessary**. In **(b != 0) && (a / b > 2)**, if **b** is 0 the division is never attempted, which prevents a crash.

23.4 Assignment Operators

The simple assignment operator **=** stores the value of the right-hand expression in the variable on the left. C also provides **compound (shorthand) assignment operators** that combine an arithmetic operation with assignment.

Shorthand	Equivalent to	Example (a = 10)	a becomes
a += 5	a = a + 5	a += 5	15
a –= 5	a = a – 5	a –= 5	5
a *= 5	a = a * 5	a *= 5	50
a /= 5	a = a / 5	a /= 5	2
a %= 3	a = a % 3	a %= 3	1

Assignment is right-to-left, so **a = b = c = 0;** sets all three to 0 (c first, then b, then a). An assignment is itself an expression whose value is the value assigned.

23.5 Increment and Decrement Operators

++ adds 1 to a variable and **−−** subtracts 1. They are unary and work only on variables (not on constants or expressions). Each can be written **before** the variable (**prefix**: **++i**) or **after** it (**postfix**: **i++**). On their own, **++i** and **i++** both simply increase **i** by 1. The difference appears when they are used **inside a larger expression**:

Form	Name	What happens	Example (i = 5)	Result
++i	Pre-increment	First increase i, **then** use the new value	x = ++i;	i = 6, x = 6
i++	Post-increment	First use the current value, **then** increase i	x = i++;	i = 6, x = 5
−−i	Pre-decrement	First decrease i, then use it	x = −−i;	i = 4, x = 4
i−−	Post-decrement	First use i, then decrease it	x = i−−;	i = 4, x = 5

23.6 Conditional (Ternary) Operator

The conditional operator **? :** is a compact form of **if-else** that produces a value. Syntax: **condition ? expression1 : expression2**. If the condition is true the whole expression takes the value of expression1, otherwise the value of expression2.

```
max = (a > b) ? a : b;          /* larger of a and b */
printf("%s", (n % 2 == 0) ? "Even" : "Odd");
grade = (marks >= 40) ? 'P' : 'F';
```

23.7 Bitwise Operators

Bitwise operators work on the individual **bits** of integer operands (int, char, long); they cannot be used on float or double. They are used in system programming, embedded systems, graphics and cryptography.

Operator	Meaning	Example (a = 12 = 1100_2, b = 10 = 1010_2)	Result
&	Bitwise AND (1 only if both bits are 1)	a & b = 1000_2	8
|	Bitwise OR (1 if either bit is 1)	a | b = 1110_2	14
^	Bitwise XOR (1 if bits differ)	a ^ b = 0110_2	6
~	Bitwise NOT / one's complement (unary)	~a	−13
<<	Left shift (multiply by 2 for each shift)	a << 2 = 110000_2	48
>>	Right shift (divide by 2 for each shift)	a >> 2 = 11_2	3

Do not confuse **&** (bitwise AND) with **&&** (logical AND), or **|** with **||**. **5 & 3** is 1 (0101 AND 0011 = 0001), while **5 && 3** is 1 because both are non-zero; but **4 & 3** is 0 whereas **4 && 3** is 1.

23.8 Special Operators

Operator	Name	Use
sizeof	Size-of operator	Gives the size in bytes of a type or variable: **sizeof(int)**, **sizeof(x)**. Evaluated at compile time; it is an operator, not a function.
,	Comma operator	Evaluates expressions from left to right; the value of the whole is the value of the rightmost. **x = (a = 3, b = 4, a + b);** gives x = 7. Common in **for** loops: **for (i = 0, j = 10; i < j; i++, j--)**.
&	Address-of operator	Gives the memory address of a variable: **&x**. Used with scanf: **scanf("%d", &x);**.
*	Indirection (dereference)	Accesses the value at an address held by a pointer: ***p**. (Pointers are covered in a later unit.)
[ ]	Subscript	Accesses an array element: **marks[3]**.
. and –>	Member access	Access members of a structure: **student.name**, **ptr->name**.
( )	Function call / grouping	**sqrt(x)**; also groups sub-expressions: **(a + b) * c**.

23.9 Program: Arithmetic, Relational and Logical Operators

Program 23.1 – operators1.c

```c
#include <stdio.h>

int main()
{
   int a = 17, b = 5;

   printf("a = %d, b = %d\n", a, b);
   printf("a + b = %d\n", a + b);
   printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d   (integer division)\n", a / b);
    printf("a %% b = %d   (remainder)\n", a % b);
    printf("a / 5.0 = %.2f (real division)\n", a / 5.0);

    printf("a > b  is %d\n", a > b);
    printf("a == b is %d\n", a == b);
    printf("a != b is %d\n", a != b);

    printf("(a > 10) && (b > 10) is %d\n", (a > 10) && (b > 10));
    printf("(a > 10) || (b > 10) is %d\n", (a > 10) || (b > 10));
    printf("!(a > 10) is %d\n", !(a > 10));
    return 0;
}
```

Output

```
a = 17, b = 5
a + b = 22
a - b = 12
a * b = 85
a / b = 3   (integer division)
a % b = 2   (remainder)
a / 5.0 = 3.40 (real division)
a > b  is 1
a == b is 0
a != b is 1
(a > 10) && (b > 10) is 0
(a > 10) || (b > 10) is 1
!(a > 10) is 0
```

Explanation

- **Line 11**: both operands are **int**, so **17 / 5** gives the integer quotient 3. **Line 13**: **5.0** is a double, so the division is done in double and gives 3.40.
- **Line 12**: to print a literal **%** sign with printf you must write **%%**.
- **Lines 15–17**: relational operators produce 1 (true) or 0 (false), which **%d** prints as numbers.
- **Lines 19–21**: **a > 10** is true (1) and **b > 10** is false (0); AND gives 0, OR gives 1, NOT of true gives 0.

23.10 Program: Increment, Decrement, Assignment and Conditional Operators

Program 23.2 – operators2.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5     int i = 5, x, y;
6     int n = 10, larger;
7
8     x = i++;       /* post-increment: x gets 5, then i becomes 6 */
9     y = ++i;       /* pre-increment : i becomes 7, then y gets 7 */
10     printf("x = %d, y = %d, i = %d\n", x, y, i);
11
12     n += 5;        /* n = n + 5  -> 15 */
13     n *= 2;        /* n = n * 2  -> 30 */
14     n %= 7;        /* n = n % 7  -> 2  */
15     printf("n = %d\n", n);
16
17     larger = (x > y) ? x : y;
18     printf("Larger of x and y = %d\n", larger);
19     printf("%d is %s\n", i, (i % 2 == 0) ? "even" : "odd");
20
21     printf("12 & 10 = %d, 12 | 10 = %d, 12 ^ 10 = %d\n", 12 & 10, 12 | 10, 12 ^ 10);
22     printf("1 << 3 = %d, 16 >> 2 = %d\n", 1 << 3, 16 >> 2);
23     return 0;
24  }
```

Output

```
x = 5, y = 7, i = 7
n = 2
Larger of x and y = 7
7 is odd
12 & 10 = 8, 12 | 10 = 14, 12 ^ 10 = 6
1 << 3 = 8, 16 >> 2 = 4
```

Explanation

- **Line 8**: **x = i++** uses the old value of **i** (5) for **x** and then makes **i** 6. **Line 9**: **y = ++i** first makes **i** 7 and then gives that value to **y**.
- **Lines 12–14**: the shorthand operators change **n** step by step: 10 → 15 → 30 → 2 (30 % 7 = 2).
- **Line 17**: the conditional operator picks the larger of **x** (5) and **y** (7).
- **Line 18**: **i** is 7, **7 % 2** is 1, so the condition is false and "odd" is printed.
- **Lines 20–21**: bitwise results as worked out in Section 23.7; **1 << 3** = $1 \times 2^3 = 8$; **16 >> 2** = $16 \div 2^2 = 4$.

Key points

- Arithmetic: + − * / %. Integer / integer gives an integer; % works only on integers.
- Relational and logical operators give 1 (true) or 0 (false). Non-zero is true.
- = assigns, == compares. && || are logical; & | are bitwise.
- ++i uses the new value; i++ uses the old value inside an expression.
- ? : is the only ternary operator; sizeof is an operator, not a function.

24. Expressions and Expression Evaluation

24.1 What is an Expression?

An **expression** is a combination of operands (variables, constants, function calls) and operators that is **evaluated to produce a single value**. **5**, **x**, **a + b**, **(a + b) * c / 2**, **x > y**, **sqrt(16.0)** and **n = 10** are all expressions. An expression followed by a semicolon becomes an **expression statement**.

Type of expression	Produces	Examples
Constant expression	A constant value known at compile time	**10 * 2**, **'A' + 1**
Arithmetic expression	A numeric value	**a + b * c**, **x / 2.0**, **n % 10**
Relational expression	1 or 0	**a > b**, **marks >= 40**
Logical expression	1 or 0	**(a > 0) && (b > 0)**, **!found**
Assignment expression	The value assigned	**x = 5**, **y += 2**
Conditional expression	One of two values	**(a > b) ? a : b**
Bitwise expression	An integer	**a & 0x0F**, **x << 2**
Mixed-mode expression	Depends on the operand types	**5 / 2.0**, **i + 3.5**

24.2 How an Expression is Evaluated

When an expression has more than one operator, C must decide the order in which to apply them. Three rules decide this:

1. **Parentheses first** – anything inside ( ) is evaluated before anything outside; innermost parentheses first.
2. **Precedence** – operators with higher precedence are applied before operators with lower precedence (e.g. * before +). See Chapter 25.
3. **Associativity** – when two operators of the **same** precedence appear side by side, associativity decides whether they are applied left-to-right or right-to-left.

Worked example 1

Evaluate **x = 10 + 3 * 2 - 8 / 4;**

```
Step 1: * and / have higher precedence than + and -; they associate left to right.
      3 * 2 = 6        ->  x = 10 + 6 - 8 / 4
      8 / 4 = 2        ->  x = 10 + 6 - 2
Step 2: + and - have the same precedence, left to right.
      10 + 6 = 16      ->  x = 16 - 2
      16 - 2 = 14      ->  x = 14
Step 3: = assigns 14 to x.
```

Worked example 2 (parentheses change the result)

Evaluate **y = (10 + 3) * (2 - 8) / 4;**

```
(10 + 3) = 13,   (2 - 8) = -6     ->  y = 13 * -6 / 4
13 * -6 = -78                     ->  y = -78 / 4
-78 / 4 = -19  (integer division truncates toward zero)
y = -19
```

24.3 Integer Expressions and Integer Division

When **all** the operands are integers the whole expression is evaluated in integer arithmetic and the result is an integer. The fractional part of a division is **truncated** (cut off, not rounded). This is the single most common source of wrong answers in beginners' programs.

Expression	Result	Reason
7 / 2	3	Integer division; 0.5 is discarded
-7 / 2	-3	Truncation toward zero (C99 rule)
7 % 2	1	Remainder
7 / 2 * 2	6	(7 / 2) = 3, then 3 * 2 = 6; not 7
1 / 2 * 10	0	1 / 2 = 0 first, then 0 * 10
10 * 1 / 2	5	10 * 1 = 10 first, then 10 / 2 = 5; order matters
7 / 2.0	3.5	One operand is double, so real division
(float) 7 / 2	3.5	7 is converted to float first (see Chapter 26)

24.4 Real (Floating-point) and Mixed-mode Expressions

If all operands are **float** or **double** the arithmetic is done in floating point. A **mixed-mode expression** contains both integer and real operands; C automatically converts the integer to real and the result is real. Only the operands of **each individual operator** are considered, not the whole expression: in **5 / 2 + 1.5** the **5 / 2** is done as integer division (2), and only then is 2 converted to 2.0 and added to 1.5, giving 3.5 and not 4.0.

24.5 Program: Expression Evaluation

Program 24.1 – expressions.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5     int a = 10, b = 3, c = 2;
6     float f = 5.0f;
7     int r1, r2, r3;
8     float r4, r5;
9
10    r1 = a + b * c;           /* 10 + (3 * 2)  = 16 */
11    r2 = (a + b) * c;         /* (13) * 2      = 26 */
12    r3 = a / b * c + a % b;   /* (10/3)*2 + 1  = 3*2 + 1 = 7 */
13    r4 = a / b;               /* integer 3 stored in float -> 3.0 */
14    r5 = a / f;               /* 10 / 5.0 = 2.0 (real division) */
15
16    printf("r1 = %d\n", r1);
17    printf("r2 = %d\n", r2);
18    printf("r3 = %d\n", r3);
19    printf("r4 = %.2f\n", r4);
20    printf("r5 = %.2f\n", r5);
21    printf("5 / 2 + 1.5 = %.2f\n", 5 / 2 + 1.5);
22    printf("5 / 2.0 + 1.5 = %.2f\n", 5 / 2.0 + 1.5);
23    return 0;
24 }
```

Output

```
r1 = 16
r2 = 26
r3 = 7
r4 = 3.00
r5 = 2.00
5 / 2 + 1.5 = 3.50
5 / 2.0 + 1.5 = 4.00
```

Explanation

- **Line 10**: ***** before **+**, so **b * c** = 6 is computed first, giving 16. **Line 11**: parentheses force the addition first, giving 26.
- **Line 12**: **/**, ***** and **%** share the same precedence and go left to right: **10 / 3** = 3, **3 * 2** = 6, **10 % 3** = 1, and finally **6 + 1** = 7.
- **Line 13**: **a / b** is an integer expression (3) even though the result is stored in a float variable; the conversion to 3.0 happens **after** the division.
- **Line 14**: **f** is float, so 10 is converted to 10.0 and real division gives 2.0.
- **Lines 21–22**: show the mixed-mode rule; only **5 / 2.0** gives real division.

24.6 Order of Evaluation of Operands (a caution)

Precedence decides which operator is applied first, but C does **not** specify the order in which the operands of most binary operators are evaluated. Expressions that change a variable and use it again in the same expression, such as **i = i++ + ++i;** or **printf("%d %d", i, i++);**, have **undefined behaviour**: different compilers give different answers. Never write such expressions; change a variable only once per statement.

Key points

- Expression = operands + operators → one value.
- Evaluation order: parentheses → precedence → associativity.
- int op int = int (division truncates); if either operand is real the result is real; conversion happens operator by operator.
- Do not modify the same variable twice in one expression (undefined behaviour).

25. Operator Precedence and Associativity

25.1 Precedence

Precedence is the rule that decides **which operator is applied first** when an expression contains different operators. Operators with higher precedence are evaluated before operators with lower precedence. Multiplication has higher precedence than addition, so **2 + 3 * 4** is 14, not 20. This follows the BODMAS rule from mathematics, extended to all the operators of C.

25.2 Associativity

Associativity is the rule that decides the order when two or more operators of the **same precedence** appear in an expression. Most operators associate **left to right**: **100 / 10 / 2** is **(100 / 10) / 2** = 5, not 100 / 5 = 20. A few operators associate **right to left**: unary operators, the conditional operator and all assignment operators. So **a = b = 5** is **a = (b = 5)**.

25.3 Precedence and Associativity Table

The table lists operators from **highest** precedence (level 1) to **lowest** (level 15). Operators in the same row have equal precedence. This table is important for the examination.

Table 25.1 – Precedence (high to low) and associativity of C operators

Level	Operators	Description	Associativity
1	( ) [ ] . -> x++ x--	Function call, array subscript, member access, postfix increment/decrement	Left to right
2	++x --x + - ! ~ (type) * & sizeof	Prefix increment/decrement, unary plus/minus, logical NOT, bitwise NOT, type cast, dereference, address-of, sizeof	Right to left
3	* / %	Multiplication, division, modulus	Left to right
4	+ -	Addition, subtraction	Left to right
5	<< >>	Bitwise shift left, right	Left to right
6	< <= > >=	Relational (less / greater)	Left to right
7	== !=	Equality, inequality	Left to right

Level	Operators	Description	Associativity
8	&	Bitwise AND	Left to right
9	^	Bitwise XOR	Left to right
10	|	Bitwise OR	Left to right
11	&&	Logical AND	Left to right
12	||	Logical OR	Left to right
13	? :	Conditional (ternary)	Right to left
14	= += −= *= /= %= &= |= ^= <<= >>=	Assignment operators	Right to left
15	,	Comma	Left to right

Easy way to remember the order

- Postfix → Unary → Arithmetic (* / % before + −) → Shift → Relational → Equality → Bitwise (& ^ |) → Logical (&& then ||) → Conditional → Assignment → Comma.
- **Unary operators** beat binary ones; **arithmetic** beats **relational**; **relational** beats **logical**; **assignment** is almost last; **comma** is the lowest of all.
- Right-to-left associativity: **unary, conditional, assignment**. Everything else is left to right.

25.4 Worked Examples

Expression	How it is evaluated	Value
2 + 3 * 4	* first: 3 * 4 = 12; then 2 + 12	14
(2 + 3) * 4	Parentheses first: 5 * 4	20
10 − 4 − 3	Left to right: (10 − 4) − 3 = 6 − 3	3
100 / 10 / 2	Left to right: (100 / 10) / 2 = 10 / 2	5
2 * 3 % 4	Same level, left to right: (2 * 3) % 4 = 6 % 4	2
−5 * 2	Unary minus first: (−5) * 2	−10
5 > 3 + 1	+ before >: 5 > 4	1
5 > 3 && 2 > 4	Relational before &&: 1 && 0	0
!5 + 1	! is unary (higher): (!5) + 1 = 0 + 1	1
a = b = c = 7	Right to left: c = 7, then b = 7, then a = 7	7
x = 5 > 3 ? 10 : 20	> before ? : ; ? : before = → x = (5 > 3) ? 10 : 20	10
1 + 2 == 3	+ before ==: 3 == 3	1
5 & 3 == 3	== before &: 5 & (3 == 3) = 5 & 1	1
a += b *= 2 (a = 3, b = 4)	Right to left: b = 8, then a = 3 + 8	11

The last few rows show two well-known traps. In **5 & 3 == 3**, the programmer probably wanted **(5 & 3) == 3** (which is 0), but because **==** has higher precedence than **&**, C computes **5 & (3 == 3)** = **5 & 1** = 1. The same problem arises with **&&**, **||** and the relational operators. **Rule of thumb: when in doubt, use parentheses.** They cost nothing and make the meaning clear to human readers.

25.5 Program: Precedence and Associativity

Program 25.1 – precedence.c

```c
#include <stdio.h>

int main()
{
   int a = 5, b = 3, c = 2, r;

   printf("a + b * c       = %d\n", a + b * c);       /* 5 + 6 = 11 */
   printf("(a + b) * c     = %d\n", (a + b) * c);     /* 8 * 2 = 16 */
   printf("a - b - c       = %d\n", a - b - c);       /* (5-3)-2 = 0 */
   printf("a / b * c       = %d\n", a / b * c);       /* (5/3)*2 = 1*2 = 2 */
   printf("a %% b + c       = %d\n", a % b + c);      /* 2 + 2 = 4 */
   printf("a > b && b > c   = %d\n", a > b && b > c);  /* 1 && 1 = 1 */
   printf("a > b == c      = %d\n", a > b == c);      /* (5>3)==2 -> 1==2 = 0 */
   printf("!a + b          = %d\n", !a + b);          /* 0 + 3 = 3 */
   printf("-a * b          = %d\n", -a * b);          /* -15 */

   r = a > b ? a : b;                                 /* 5 */
   printf("r = a > b ? a : b   -> r = %d\n", r);
   r = a = b = 9;                                     /* right to left */
   printf("r = a = b = 9       -> r = %d, a = %d, b = %d\n", r, a, b);
   return 0;
}
```

Output

```
a + b * c       = 11
(a + b) * c     = 16
a - b - c       = 0
a / b * c       = 2
a % b + c       = 4
a > b && b > c   = 1
a > b == c      = 0
!a + b          = 3
-a * b          = -15
r = a > b ? a : b   -> r = 5
r = a = b = 9       -> r = 9, a = 9, b = 9
```

Explanation

- **Line 7 vs 8**: **`*`** binds tighter than **`+`**; parentheses override it.
- **Line 9**: **`–`** is left associative, so subtraction is done from the left.
- **Line 10**: **`/`** and **`*`** have the same precedence, so **`5 / 3`** (= 1) is done first, then **`1 * 2`**.
- **Line 13**: relational **`>`** (level 6) is evaluated before equality **`==`** (level 7), so the expression becomes **`1 == 2`**, which is 0.
- **Line 14**: **`!`** is a unary operator with higher precedence than **`+`**, so **`!a`** (= 0) is computed first.
- **Line 19**: chained assignment is right to left: **`b = 9`**, then **`a = b`**, then **`r = a`**.

Key points

- Precedence: which operator first (higher before lower). Associativity: order among operators of equal precedence.
- Order to remember: ( ) → unary → * / % → + − → shifts → relational → equality → & ^ | → && → || → ? : → = → comma.
- Right-to-left: unary, ? :, assignment. All others left-to-right.
- == and != have higher precedence than & | && ||; use parentheses to be safe.

26. Type Conversions

26.1 Why Conversion is Needed

An expression often mixes values of different types: an **int** added to a **float**, a **char** compared with an **int**, a **double** stored in an **int** variable. Since the CPU can only operate on two values of the **same** type, one of them must be converted. **Type conversion** (or type casting) is the process of changing a value from one data type to another. C does this in two ways: **implicitly** (automatically, by the compiler) and **explicitly** (on the programmer's instruction).

26.2 Implicit Type Conversion (Automatic / Coercion)

The compiler converts values automatically in three situations.

(a) Integer promotion

Whenever a **char** or a **short** is used in an expression it is first converted (promoted) to **int**. That is why **'A' + 1** is the integer 66 and why a **char** can be printed with **%d**.

(b) Usual arithmetic conversions in an expression

When the two operands of a binary operator have different types, the operand of the **lower** type is converted to the **higher** type, and the operation and its result are in that higher type. The ranking is shown in Figure 26.1: from **char** and **short** (lowest) through **int**, **unsigned int**, **long**, **unsigned long**, **long long** to **float**, **double** and **long double** (highest). In short: **int + float gives float; float + double gives double; int + long gives long**.

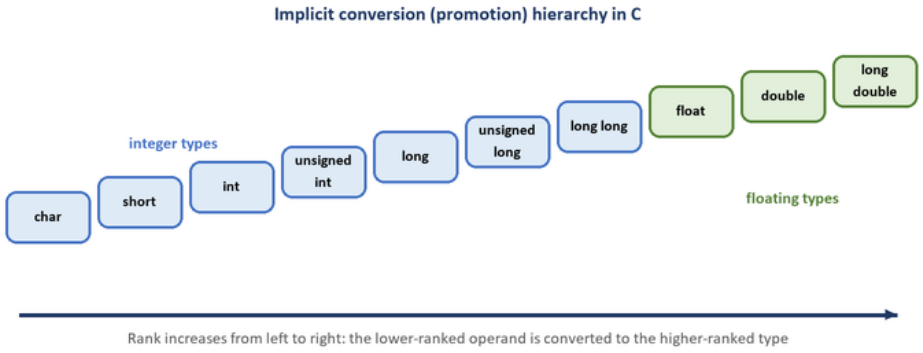


Figure 26.1 – The promotion hierarchy: the lower-ranked operand is converted to the higher-ranked type

Expression	Conversion applied	Result and type
5 + 2.5	5 (int) → 5.0 (double)	7.5 double
7 / 2	none (both int)	3 int
7 / 2.0	7 → 7.0	3.5 double

Expression	Conversion applied	Result and type
'A' + 1	'A' → 65 (int)	66 int
10 * 1.5f	10 → 10.0f	15.0 float
3.0f + 2.0	3.0f (float) → 3.0 (double)	5.0 double
−1 < 1u	−1 (int) → unsigned (a huge value)	0 (false!) – a classic trap

(c) Conversion during assignment

When a value is assigned to a variable of a different type, the value is converted to the type of the **variable on the left**, regardless of the types on the right. Converting to a **wider** type is safe (**promotion**, e.g. **int → double**). Converting to a **narrower** type may **lose information** (**demotion** or narrowing, e.g. **double → int** drops the fractional part; **int → char** keeps only the lowest 8 bits).

Statement	What happens	Value stored
int x = 7.9;	double → int: fraction truncated (not rounded)	7
float y = 5;	int → float	5.0
int z = −3.99;	truncation toward zero	−3
char c = 321;	int → char: 321 = 0x141, only 0x41 kept	65 ('A')
int k = 3000000000;	overflow: value does not fit in int	wrong (negative) value
double d = 1 / 3;	1 / 3 is computed as int (0) first, then converted	0.0 (not 0.333)

26.3 Explicit Type Conversion (Type Casting)

Sometimes the programmer wants a conversion that C would not do automatically, most often to force **real division** on two integers. This is done with the **cast operator**: **(type) expression**. The cast is a unary operator with high precedence (level 2), so it applies only to the operand immediately following it.

```
int total = 7, count = 2;
float avg;

avg = total / count;            /* 3.0  -> integer division, then converted */
avg = (float) total / count;    /* 3.5  -> total becomes 7.0f, real division */
avg = (float) (total / count);  /* 3.0  -> division done first in int, then cast */

int n = (int) 3.99;             /* 3 : fraction dropped */
char ch = (char) 66;            /* 'B' */
int r = (int) (2.5 + 0.5);      /* 3 : a simple way to round a positive number */
```

The second and third lines show the most important detail: **(float) total / count** casts **only total**, so the division is real (3.5); whereas **(float) (total / count)** casts the **result** of an integer division (3), which is too late (3.0).

26.4 Program: Implicit and Explicit Conversion

Program 26.1 – conversion.c

```c
#include <stdio.h>

int main()
{
   int   total = 7, count = 2;
   int   i;
   float avg1, avg2, f;
   char  ch = 'A';
   double d = 9.99;

    avg1 = total / count;           /* implicit: 3 -> 3.0 */
    avg2 = (float) total / count;    /* explicit cast: 3.5 */
    printf("avg1 = %.2f, avg2 = %.2f\n", avg1, avg2);

    f = ch + 1;                  /* 'A' promoted to 65, 66 -> 66.0 */
    printf("ch + 1 = %.1f\n", f);

    i = d;                       /* demotion: 9.99 -> 9 */
    printf("i = %d (from %.2f)\n", i, d);

    i = (int) (d + 0.5);            /* rounding trick: 10 */
    printf("rounded = %d\n", i);

    printf("5 / 2 = %d, 5 / 2.0 = %.1f, (double) 5 / 2 = %.1f\n",
          5 / 2, 5 / 2.0, (double) 5 / 2);
    printf("sizeof(ch + 1) = %zu bytes (promoted to int)\n", sizeof(ch + 1));
    return 0;
}
```

Output

```
avg1 = 3.00, avg2 = 3.50
ch + 1 = 66.0
i = 9 (from 9.99)
rounded = 10
5 / 2 = 2, 5 / 2.0 = 2.5, (double) 5 / 2 = 2.5
sizeof(ch + 1) = 4 bytes (promoted to int)
```

Explanation

- **Line 11**: **total / count** is **7 / 2** in integer arithmetic (3); the 3 is then converted to 3.0 for storage in **avg1**.
- **Line 12**: the cast converts **total** to 7.0f **before** the division, so real division gives 3.5.
- **Line 15**: **ch** is promoted to int (65), 1 is added, and the result 66 is converted to float 66.0 for assignment.
- **Line 18**: assigning a double to an int truncates 9.99 to 9 (no rounding).
- **Line 21**: adding 0.5 before truncating is the standard way to round a positive value: 10.49 → 10.
- **Line 26**: **sizeof(ch + 1)** is 4, not 1, proving that **ch** was promoted to **int** in the expression.

26.5 Implicit versus Explicit Conversion

Implicit conversion	Explicit conversion (casting)
Done automatically by the compiler.	Done deliberately by the programmer using (type).

Implicit conversion	Explicit conversion (casting)
Happens in mixed expressions and assignments.	Happens only where the cast is written.
Lower type is promoted to higher type (int → float).	Any conversion possible, including narrowing (float → int).
May silently lose data on assignment to a narrower type.	Makes the programmer's intention clear; loss of data is expected.
Also called coercion or automatic conversion.	Also called type casting.

26.6 Common Mistakes

- Writing **average = sum / n;** with both **sum** and **n** integers and wondering why the decimals are missing. Fix: **(float) sum / n**.
- Casting the result instead of the operand: **(float) (sum / n)** still does integer division.
- Expecting **int x = 7.9;** to round to 8; it truncates to 7.
- Comparing signed and unsigned values (**−1 < 1u** is false).
- Storing a large value in a **char** or **short** and losing the high bits.

Key points

- Implicit: char/short → int (promotion); lower type → higher type in expressions; right side → type of the left variable in assignment.
- Hierarchy: char < short < int < unsigned < long < long long < float < double < long double.
- Narrowing (double → int) truncates toward zero; can lose data.
- Explicit cast (type) expr applies only to the next operand: (float) a / b is real division; (float) (a / b) is not.

27. Common Mistakes and Exam-oriented Questions

27.1 Ten Common Beginner Mistakes in C

#	Mistake	Correct form / remedy
1	Forgetting the semicolon at the end of a statement	Every declaration and expression statement ends with ;
2	Using = instead of == in a condition: **if (x = 5)**	**if (x == 5)**
3	Missing **&** in scanf: **scanf("%d", n);**	**scanf("%d", &n);** (scanf needs the address)
4	Wrong format specifier: printing a float with %d	Match type and specifier: %d int, %f float, %lf double, %c char
5	Expecting 7 / 2 to be 3.5	Use a real operand or a cast: 7 / 2.0 or (float) 7 / 2
6	Using a variable before initialising it	Initialise: **int sum = 0;**
7	Writing **void main()**	Write **int main()** and end with **return 0;**
8	Case errors: **Printf**, **Int**, **MAIN**	C is case sensitive; keywords and library names are lowercase
9	Using a keyword as a variable name: **int float;**	Choose another identifier
10	Mismatched braces, or a semicolon after **#include** / **#define**	Directives take no semicolon; count { and }

27.2 Short-answer Questions (2 marks)

1. Define a computer. State any four characteristics.
2. Distinguish between data and information.
3. What is the difference between RAM and ROM?
4. What is cache memory? Why is it needed?
5. Define an operating system. List its four main functions.
6. What is the fetch–decode–execute cycle?
7. Differentiate between a compiler and an interpreter (any three points).
8. What is the role of the linker? Of the loader?
9. Define an algorithm. List Knuth's five properties.
10. Draw the symbols used in a flowchart for start/stop, input/output, process and decision.
11. What is pseudo-code? Give an example.
12. What is structured programming? Name its three control structures.
13. Who developed C, when and where? Why was it developed?
14. List the six sections of a C program. Which is compulsory?
15. What are tokens? Name the six types of tokens in C.
16. State the rules for naming an identifier. Give two invalid identifiers with reasons.
17. What is the difference between **'A'** and **"A"**?
18. Differentiate between **#define** and **const**.
19. What is the output of **printf("%d", 17 / 5 * 2 + 17 % 5);**? (Answer: 8)
20. What is the difference between **++i** and **i++**?
21. What is the value of **x** after **int x = 5; x += x++ + ++x;**? (Answer: undefined behaviour; explain why.)
22. Define precedence and associativity with one example each.
23. What is type casting? Write a statement that divides two integers and gives a real result.
24. What is the difference between implicit and explicit type conversion?

27.3 Long-answer Questions (5 to 10 marks)

1. Draw and explain the block diagram of a computer, describing the function of each unit.
2. Explain the memory hierarchy with a neat diagram. Compare primary and secondary memory.
3. Classify computers based on (a) purpose, (b) type of data handled and (c) size. Explain the generations of computers.
4. Explain the functions and types of operating systems.
5. Explain the working of a CPU with its important registers and the instruction cycle.
6. Explain the generations of programming languages. Compare low-level and high-level languages.
7. With a neat diagram explain the process of converting a C source program into an executable program (preprocessor, compiler, assembler, linker, loader).
8. Explain the steps involved in developing a program. Describe the types of errors.
9. Explain the phases of the software development life cycle.
10. What is an algorithm? Write an algorithm, pseudo-code and flowchart to (a) find the largest of three numbers, (b) find the factorial of a number, (c) check whether a number is prime.
11. Explain the concept of structured programming with its advantages.
12. Explain the structure of a C program with an example program that uses all the sections.
13. Explain the basic data types in C with their size, range and format specifiers.
14. Explain the different types of constants in C with examples.

15. Explain the various operators available in C with examples of each category.
16. Explain operator precedence and associativity with the precedence table and suitable examples.
17. Explain implicit and explicit type conversion in C with example programs.
18. Write a C program to read the marks of five subjects and print the total, average and percentage. Explain each line.

27.4 Predict-the-output Practice

Work out the output by hand, then check on a compiler. Answers are given at the end.

```
/* Q1 */  int a = 10, b = 4;  printf("%d %d %.2f", a / b, a % b, (float) a / b);
/* Q2 */  int i = 3;  printf("%d %d", i++, ++i);       /* think carefully */
/* Q3 */  int x = 5;  printf("%d", x > 3 && x < 10);
/* Q4 */  printf("%d", 2 + 3 * 4 - 8 / 2 % 3);
/* Q5 */  char c = 'a';  printf("%c %d", c + 1, c + 1);
/* Q6 */  int n = 7;  printf("%d", n % 2 == 0 ? 0 : 1);
/* Q7 */  float f = 7 / 2;  printf("%.1f", f);
/* Q8 */  int p = 5, q = 2;  printf("%d", p / q * q + p % q);
/* Q9 */  printf("%d", sizeof(char) + sizeof('A'));      /* on a 4-byte-int compiler */
/* Q10 */ int m = 10;  m -= 3 * 2;  printf("%d", m);
```

Q	Output	Reason
1	2 2 2.50	10/4 = 2 (int), 10%4 = 2, cast gives real division
2	undefined	i is modified twice in one expression; the order of evaluation of printf arguments is unspecified. Common compilers print 4 5 or 3 5; never write this.
3	1	Both conditions true, so && gives 1
4	13	3*4 = 12, 8/2 = 4, 4%3 = 1, then 2 + 12 − 1 = 13
5	b 98	'a' is 97; 97 + 1 = 98 = 'b'; %c prints the character, %d the code
6	1	7 % 2 is 1, not equal to 0, so the false branch (1) is chosen
7	3.0	7 / 2 is integer division (3) before conversion to float
8	5	(5/2)*2 = 2*2 = 4; 5%2 = 1; 4 + 1 = 5
9	5	sizeof(char) = 1; a character constant 'A' has type int in C, so sizeof('A') = 4
10	4	m = m − (3*2) = 10 − 6 = 4; the right side is evaluated fully first

28. Quick Revision Sheet

One-line facts for last-minute revision. Read this section the night before the examination.

28.1 Computer Fundamentals

- Computer: electronic device performing Input → Process → Output under program control; characteristics: speed, accuracy, storage, diligence, versatility, automation.
- Classification: by purpose (general / special), by data (analog / digital / hybrid), by size (micro / mini / mainframe / super).
- Generations: 1 vacuum tubes, 2 transistors, 3 ICs, 4 microprocessors, 5 AI.
- Functional units: Input, CPU (CU + ALU + registers), Memory, Output, Secondary storage (von Neumann architecture).

- Memory hierarchy (fast → slow): Registers → Cache → RAM → SSD/HDD → Tape/optical. RAM volatile, ROM non-volatile. 1 byte = 8 bits; KB, MB, GB, TB = ×1024 each.
- OS = resource manager + interface; functions: process, memory, file, device management, UI, security; types: batch, multiprogramming, time-sharing, real-time, distributed, network, mobile; kernel = core.
- CPU cycle: Fetch → Decode → Execute; registers PC, IR, MAR, MDR, ACC, flags; speed in GHz; address / data / control buses.

28.2 Formula and Fact Card

Item	Value / rule
ASCII codes to remember	'0' = 48, 'A' = 65, 'a' = 97, space = 32, '\n' = 10, '\0' = 0
Even / odd test	n % 2 == 0 → even
Last digit of n	n % 10; remove last digit: n / 10
Largest of a, b	(a > b) ? a : b
Real average of ints	(float) sum / n
Round a positive real to int	(int) (x + 0.5)
Range of signed int (4 bytes)	−2,147,483,648 to 2,147,483,647 (-2^{31} to $2^{31} - 1$)
Range of unsigned char	0 to 255
Sum of 1 to n	n (n + 1) / 2
Compile and run (GCC)	gcc prog.c -o prog then ./prog (Linux) or prog (Windows)

Final advice for the examination

- For definitions, write the definition, one example and one diagram where possible (block diagram, memory pyramid, flowchart symbols, compilation chain).
- For differences, use a table with at least four points.
- For programs, write clean, indented code with #include, int main(), return 0, comments, and show the expected output.
- For predict-the-output questions, apply precedence and associativity step by step, and remember integer division.